

MAS-Orchestra：通过整体式编排与受控基准理解并改进多智能体推理

Zixuan Ke¹ Yifei Ming¹ Austin Xu¹ Ryan Chin² Xuan-Phi Nguyen¹ Prathyusha Jwalapuram¹
Jiayu Wang³ Semih Yavuz¹ Caiming Xiong¹ Shafiq Joty¹



项目主页: https://mas-orchestra.salesforceresearch.ai/mas_r1/



在线演示: https://mas-orchestra.salesforceresearch.ai/mas_r1/demo/



MASBench: <https://huggingface.co/datasets/Salesforce/MASBench>



代码: <https://github.com/SalesforceAIResearch/MAS-Orchestra>

摘要

多智能体系统 (multi-agent systems, MAS) 有望通过智能体间的协作提升智能水平, 但现有自动化 MAS 设计方法的实际表现仍未达到预期。这些不足源于两个关键因素: (1) 方法复杂性——智能体编排采用顺序式代码级执行, 既限制了全局系统层面的整体推理, 也难以随智能体复杂度扩展; (2) 效能不确定性——人们在尚未弄清 MAS 相较单智能体系统 (single-agent systems, SAS) 是否具有切实收益的情况下便部署了 MAS。我们提出 MAS-Orchestra, 这是一个训练时框架, 它将 MAS 编排表述为采用整体式编排的函数调用强化学习问题, 一次性生成完整的 MAS。在 MAS-Orchestra 中, 复杂且面向目标的子智能体被抽象为可调用函数; 这种抽象在隐藏内部执行细节的同时, 使模型能够对系统结构进行全局推理。为了严谨研究 MAS 在何时以及为何有益, 我们提出受控基准 MAS-BENCH, 从深度、时域、广度、并行性和

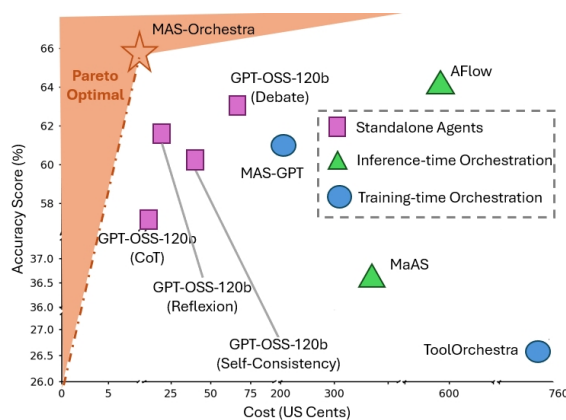


图 1. 基于 AIME24 和 GPQA 的平均准确率与总推理成本计算所得的成本-性能权衡。MAS-Orchestra 位于帕累托前沿, 以低于或接近强基线的成本取得更高准确率 (见 Section D.4)。所有训练时编排模型均使用其已发布的编排器, 而这些编排器的训练环境可能与我们的环境不同。

鲁棒性 五个维度刻画任务。我们的分析表明, MAS 的收益并非普遍存在, 而是关键取决于任务结构、验证协议以及编排器与子智能体双方的能力。以这些洞见为指导, MAS-Orchestra 在数学推理、多跳问答和基于搜索的问答等公共基准上取得了稳定提升, 同时效率比强基线高出超过 $10\times$ 。MAS-Orchestra 与 MASBENCH 共同推动了 MAS 的训练与理解, 助力实现多智能体智能。

¹Salesforce Research ² 麻省理工学院 ³ 威斯康星大学麦迪逊分校. 通讯作者: Zixuan Ke <zixuan.ke@salesforce.com>, Shafiq Joty <sjoty@salesforce.com>.

1. 引言

我们已经观察到一条清晰的演进路线: 从独立的大语言模型 (LLM), 到基于 LLM 的单智能体系统

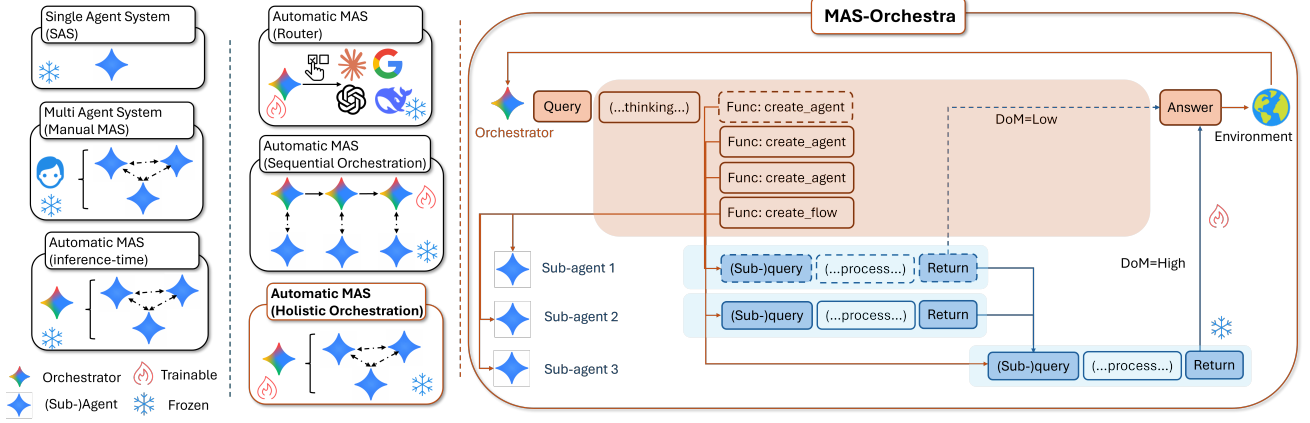


图 2. 不同范式与 MAS-Orchestra 的对比。左：推理时编排系统通常采用整体式编排，但不进行训练。MAS-Orchestra 属于自动化 MAS，并将问题表述为采用整体式编排的函数调用强化学习问题。右：当 DoM 被设为低（虚线）时，系统至多实例化一个智能体；当 DoM 为高时，智能体数量不受限制。函数调用协议与解析器的细节见附录 G 和附录 H。

(SAS)，再到近来的多智能体系统 (MAS) (Ke et al., 2025a)。这一转变反映了现代任务日益增长的复杂性；这些任务往往需要持续推理、任务分解以及与其他专门智能体交互。早期 MAS 设计大多由人工构建，采用辩论等固定交互模式 (Du et al., 2023)。近年来，研究重心逐渐转向自动化 MAS 设计：系统结构、智能体角色与交互关系均动态生成，因为人工指定不仅耗费大量人力，也难以扩展到新问题或新领域。

尽管自动化 MAS 已展现出很强的实证潜力，现有方法在几个重要方面仍存在局限：(1) **问题表述**：多数已有工作 (Ke et al., 2025b; Nie et al., 2025; Hu et al., 2024) 采用可执行代码来表述智能体编排。基于代码的表述虽然灵活，但会带来显著开销；当子智能体及其连接关系变得更加复杂时（例如执行多轮搜索的智能体），这种方法也难以扩展，因为编排器在编排过程中往往需要分析、甚至复现完整的子智能体代码。因此，这些方法通常局限于相对简单的子智能体，例如基于思维链 (Chain-of-Thought, CoT) (Wei et al., 2022) 和自洽思维链 (Self-Consistency, SC) (Wang et al., 2023) 的子智能体。(2) **训练**：许多现有方法高度依赖推理时自我改进或启发式搜索 (Zhang et al., 2025c; Liu et al., 2024)。缺乏显式训练目标时，这种适应过程可能不稳定、优化方向错误，而且计算成本高昂。尽管近期一些研究已将训练引入 MAS 设计 (Dang et al., 2025b; Chi et al., 2025)，它们通常仍

将编排建模为顺序决策过程，即每次增量添加一个组件，并通过多轮强化学习进行优化。这种范式在部分场景下有效，却会引入大量训练开销并鼓励局部的逐步优化，使编排器难以从全局视角推理 MAS 结构；(3) **智能体扩展原则** (Li et al., 2024; Qian et al., 2025b)。尽管 MAS 正被迅速采用，目前仍没有原则性的定量框架能够预测增加子智能体何时会提升性能、何时反而会降低性能。因此，实践者往往依赖启发式方法，这既限制了科学理解，也阻碍了实际部署。多智能体协作何时真正优于更简单的单智能体方案、何时只是增加复杂性，仍不明确 (Kim et al., 2025; Huang et al., 2024)。

我们认为，一个有效的自动化 MAS 应满足三个核心要求：(1) 应有一个明确的“**MAS 程度 (DoM)**”概念，用以刻画特定任务适合采用多大程度的多智能体协作，因为并非所有问题都能从 MAS 中获益。例如，我们的先导实验 (Section 6) 表明，即便是 AIME 中颇具挑战性的数学任务，往往也很少受益于多智能体协作；(2) **编排机制应当灵活且可扩展**。即使子智能体高度复杂，该问题表述也应保持有效。特别是，编排器 LLM 应当只关注高层推理与系统设计：它的职责是决定何时调用哪些子智能体，以及如何连接它们，而不是复现子智能体自身的内部行为；(3) **子智能体应由自己的目标定义，因此应拥有自己的上下文、工具和工作流** (Anthropic, 2024b)。多数现有方法要么在 LLM 空间中定义子智能体 (Jin

et al., 2025; Estornell et al., 2025; Dang et al., 2025b), 通过切换不同骨干模型来区分子智能体, 这会让编排器退化为单纯的 LLM 路由器; 要么在提示词模板空间中定义子智能体, 使不同子智能体之间只有表层的提示词差异 (例如在辩论中, 不同角色往往仅通过修改角色描述来实例化: “你是一名批评者”与“你是一名支持者”)。此类定义忽略了工具使用与 workflow, 而这些同样是让子智能体围绕其目标形成专业能力的关键因素。

本文提出 **Mas-Orchestra**, 它将 MAS 编排表述为**函数调用强化学习 (RL) 问题**。在这一表述下, 复杂度任意且面向目标的子智能体 (例如推理智能体和搜索智能体) 被封装为可调函数, 其内部机制则被抽象隐藏。编排器只需通过调用 `create_agent` 和 `create_flow` 两个原语函数, 决定何时实例化子智能体、创建哪些子智能体, 以及如何连接它们。我们还在问题表述中引入了 **DoM**, 用户可将其配置为低 (至多使用一个子智能体) 或高。基于这一表述, 我们不再将 MAS 设计视为一系列决策, 而是采用**整体式编排**, 由编排器在**单个决策步骤**中生成完整编排 (对比见 Figure 2)。这种设计使编排器能够联合推理完整系统配置, 而不是优化一连串局部决策。因此, 学习到的编排反映的是全局协作模式, 而不是执行顺序造成的影响。从学习角度看, 它比多步方法更容易优化: 多步方法会诱导局部的逐步优化, 并受到长时域信用分配和误差传播的影响。因此, 随着 MAS 复杂度上升, 该设计能带来更稳定的训练和更好的可扩展性; 其性能优于顺序式编排基线 (Section 6) 也证明了这一点。它还带来了显著的效率提升 (超过 $10\times$), 因而位于帕累托前沿, 如图 Figure 1 所示。

为了定量分析 MAS 在何时以及为何优于单智能体系统, 我们提出受控基准 **MasBENCH**。它从深度、时域、广度、并行性和鲁棒性五个维度刻画问题, 从而能够系统且受控地评估 MAS 行为。我们的分析得出了若干关键洞见。例如, 我们发现多智能体协作的收益高度依赖于任务结构和底层 LLM 的能力。在这些洞见的指导下, 我们在公共基准上开展实验, 并证明恰当的 MAS 配置能够带来稳定的性能提升。

方法	训练	面向目标的子智能体	整体式编排	MAS 程度	MAS 与 SAS 对比分析
MAS-Zero (Ke et al., 2025b)	✗	✗	✓	✗	✗
AFlow (Zhang et al., 2025c)	✗	✗	✗	✗	✗
DyLAN (Liu et al., 2023)	✗	✗	✗	✗	✗
MAS-GPT (Ye et al., 2025)	SFT	✗	✓	✗	✗
CoRL (Jin et al., 2025)	RL	✗	✗	✗	✗
Puppet (Dang et al., 2025a)	RL	✗	✗	✗	✗
xRouter (Qian et al., 2025a)	RL	✗	✗	✗	✗
W4S (Nie et al., 2025)	RL	✗	✗	✗	✗
ToolOrchestra (Su et al., 2025)	RL	✓	✗	✗	✗
Mas-Orchestra	RL	✓	✓	✓	✓

表 1. 主流自动化 MAS 方法对比。

总之, 本文的主要贡献如下:

- 我们为 MAS 编排提出一种新颖、可扩展且有效的强化学习表述, 其中包含明确的 *DoM* 概念, 以及封装复杂子智能体的函数调用抽象。
- 我们提出专为 MAS 评估设计的受控基准 **MasBENCH**, 可系统地实证分析 MAS 何时优于单智能体系统、何时并非如此。据我们所知, 这是首个专门评估 MAS 收益的基准。
- 借助 **MasBENCH**, 我们从三个分析方向研究广泛 MAS 配置下的收益, 覆盖不同模型规模与家族中的三种编排器设置和五种子智能体设置。
- 在上述分析洞见的指导下, 我们证明本方法在数学、多跳问答和多步搜索问答等公共基准上表现强劲, 同时效率比强基线高出超过 $10\times$ 。

2. 相关工作

2.1. 从单智能体系统到多智能体系统

尽管智能体性通常被视为一个连续谱 (Kapoor et al., 2024), 清楚地区分单智能体系统 (SAS) 与多智能体系统 (MAS) 仍是一项基础工作。我们将 SAS 定义为仅有一个推理中心且只对应一个目标的系统: 所有感知、规划和行动都在由单个 LLM 实例控制的顺序控制循环内执行, 以实现该目标。这一定义包括采用工具使用 (Yao et al., 2023)、自我反思 (Madaan et al., 2024) 或 CoT 推理的系统。相比之下, MAS 由多个具有各自目标的 LLM 智能体组成; 它们通过消息传递、共享记忆或显式编排协议等结构化通信机制交互 (Xi et al., 2023)。每个智能体都可以维护自己的上下文、目标和工具, 系统级行为则通过智能体

间的协作以集体推理形式涌现，而非源自单一的整体推理过程。

根据不同语境下对子智能体的定义，MAS 设计还与 LLM 路由器 (Su et al., 2025; Zhang et al., 2025b) (将 LLM 视为子智能体) 及技能 (Skills) (Anthropic, 2024a) (将技能视为子智能体) 相关。尽管已有工作研究人工 MAS 设计和自动化推理时编排 (见 Appendix E)，考虑到**训练时编排**日益受到关注且与 MAS-Orchestra 直接相关，我们将讨论重点放在这一方向。

2.2. 自动化训练时编排

这一研究方向通过直接训练编排器来避免昂贵的推理时适应。该类别中的多数现有方法采用顺序式编排，并使用多步强化学习进行优化 (Su et al., 2025; Nie et al., 2025)。MAS-GPT (Ye et al., 2025) 是一个例外，它不使用强化学习来训练编排器；然而，其报告性能甚至低于推理时适应方法 (Ke et al., 2025b)，说明它在学习有效的系统级协作方面存在局限。据我们所知，MAS-Orchestra 是首个执行训练时整体式编排的方法，其中元智能体被训练成在单个决策步骤中生成完整的多智能体系统。这种表述使 MAS-Orchestra 的编排器能够在规划层面而非执行轨迹上进行推理，从而实现子智能体间的全局协作，避免中间状态引起的误差累积，并让训练目标直接对齐最终任务性能。表 1 系统对比了 MAS-Orchestra 与具有代表性的推理时、训练时 MAS 设计方法。结果表明，MAS-Orchestra 是唯一同时支持面向目标的子智能体、整体式编排、可配置 DoM，以及旨在理解 MAS 何时及如何优于 SAS 的受控实验的方法。

3. MAS-Orchestra 框架

3.1. 提出的 MAS 问题表述

令 $\mathcal{D} = \{(x_i, y_i)\}$ 表示一个数据集，其中 $x_i \in \mathcal{X}$ 为任务输入， $y_i \in \mathcal{Y}$ 为真实答案。目标是从 $\mathcal{D}$ 中学习策略 π_θ ，将任意任务输入 x 映射为潜在编排 a ； **a 编码一个完整的 MAS 编排**，并作为集体推理过程得到答案 $\hat{y}$ 。

令 $m \in \{\text{Low}, \text{High}\}$ 表示用户选择的 **DoM 级别**。形式化地，编排按如下方式生成：

$$a \sim \pi_\theta(\cdot \mid x, m). \quad (1)$$

DoM 变量 m 约束允许的编排空间，例如子智能体数量，以及是否允许显式的智能体间拓扑。在本文中，低 DoM 表示编排器至多只能实例化一个子智能体，并且不能使用显式的智能体间拓扑。¹需要特别指出的是，低 DoM 不同于 SAS。在低 DoM 下，编排器仍须决定是委派整个任务、委派子任务，还是自行完成任务；它还必须决定使用哪个子智能体以及如何配置该子智能体（而在高 DoM 下，编排器还需确定智能体间拓扑）。编排 a 规定实例化哪些子智能体、信息如何在它们之间流动，以及如何聚合它们的输出以得到答案。

将面向目标的子智能体视为函数。我们把每个子智能体建模为面向目标的黑盒函数，调用它来解决特定子任务或实现指定目标。编排器只能看到子智能体的签名（即名称与可配置参数），其内部推理和执行细节均被抽象隐藏。整个编排完全由函数调用指定，其中既包括子智能体，也包括它们之间的连接。预先定义、基于规则的确定性解析器 f 解释编排规格，²根据分配的目标实例化相应子智能体并执行它们，最终给出预测：

$$\hat{y} = f(x, a). \quad (2)$$

作为首项训练整体式编排的工作，MAS-Orchestra 在单个决策步骤中生成整个编排。编排器看不到执行期间产生的中间状态或部分结果；编排质量只依据最终输出评估。Figure 2 展示了这一过程。

3.2. 面向整体式编排的强化学习

我们根据最终答案是否正确来定义任务级奖励（根据 Table 2 中的验证协议，也可能纳入子任务答案的正确性）：

$$R(x, y, \hat{y}) = \begin{cases} 1, & \text{若 } \hat{y} = y, \\ 0, & \text{否则.} \end{cases} \quad (3)$$

¹提示词与示例见 Appendix G。

²细节见 Appendix H。

学习目标是最大化期望奖励：

$$\max_{\theta} \mathbb{E}_{(x,y) \sim \mathcal{D}} \mathbb{E}_{a \sim \pi_{\theta}(\cdot | x, m)} [R(x, y, f(x, a))] \quad (4)$$

我们使用组相对策略优化（Group Relative Policy Optimization, GRPO）（Shao et al., 2024）来优化策略。该方法将每个采样编排与同组其他候选编排比较，并据此更新策略。对于每个输入 x ，我们采样一组 K 个编排 $\{a_i\}_{i=1}^K \sim \pi_{\theta}(\cdot | x, m)$ ，并获得对应奖励 $\{R_i\}_{i=1}^K$ 。随后利用这些组内样本与奖励构造组相对裁剪策略梯度目标（详见 Appendix F）。

4. 面向 MAS 的分析框架

为了定量评估 MAS-Orchestra，我们采用两阶段评估策略：（1）通过受控实验比较 MAS 与 SAS（Section 5）；（2）在公共基准上评估（Section 6）。本节建立支撑第一阶段的分析框架。关键在于，据我们所知，此前尚无专门的评估框架与基准，能够在受控环境下严谨评估二者的差异。

4.1. 五维评估框架

为了在单智能体系统与多智能体系统之间开展受控比较，我们将评估建立在“协作复杂度如何超越单一推理线程而扩展”这一问题上。我们把 SAS 视为智能体计算的最小单元，而 MAS 通过多个智能体之间的显式协作扩展这一单元。MAS 的性能收益可能来自有效的智能体协作与专业化，也可能来自集成效应带来的有效计算量增加。关键在于，这些收益能在多大程度上实现取决于两个因素：（1）**任务的内在结构**，它决定如何分解与协调推理；（2）**验证协议**，它决定是否以及如何显式生成、复用或验证中间子任务输出。

为了以受控方式研究这些影响，我们假设每个任务都可以隐式或显式地分解为有限个**子任务**，它们构成推理与协作的基本单元。随后，我们设计了**五个评估维度**。在任务结构方面，我们考虑深度、时域、广度与并行性，分别刻画不同的依赖、可分解性与协作模式。在验证协议方面，深度、广度和并行性主要在**最终输出**处评估正确性，而时域同时评估

中间输出和**最终输出**，对应需要传递并复用中间结果的长时域推理。最后，我们引入鲁棒性，衡量系统在存在**对抗性或错误中间信息**时进行可靠推理的能力。Table 2对五个维度进行了比较（完整示例见 Section A.1.2）。对于每个维度，相应数值表示满足其定义的子任务数量，因而可以近似反映任务复杂度；数值越大，复杂度越高。这些维度共同施加不同的推理与协作需求，使我们能够系统比较 SAS 与 MAS。

4.2. MASBENCH 基准

遵循上述五维评估框架，我们构建了新基准 **MAS-BENCH**，以支持可复现评估与后续研究。MASBENCH 对五个评估维度进行了标准化实例化，使研究者能够以受控且可比较的方式分析不同任务结构与验证协议如何影响 SAS 和 MAS 的相对表现。

基准构建。 MASBENCH 中的每个实例由一个问题及其依赖图定义；图中节点对应子任务，边编码子任务依赖。五个评估维度均直接从该依赖图实例化。具体而言，深度、时域、广度和并行性由依赖链长度或最大入度等图结构属性决定。为了实例化鲁棒性维度，我们为每个子任务加入一条**简短的对抗性注释**，其中包含来自上游子任务的错误信息。具体地，子任务描述 s_i 被扩充为 $\tilde{s}_i = (s_i, \text{note}_i)$ ，其中 note_i 的形式为：“注：采用信息前请先验证——/上一个子任务的错误答案/”（见 Table 2）。

MASBENCH 的源数据集。 我们主要从合成数据生成器 iGSM（数学）（Ye et al., 2024）中整理实例与依赖图。iGSM 能生成结构复杂度（深度、时域、广度和并行性）可控的依赖图，并自动推导相应的自然语言问题。为了避免数据泄漏，我们在生成时使用不同哈希值，确保训练集与测试集互不重叠，且不存在模板级重叠。尽管鲁棒性维度适用于所有结构化任务类型，实践中我们只将其应用于深度等于 4 的任务。为此，我们将 iGSM 子任务指令与 RULER 基准中的大海捞针（needle-in-a-haystack, NIAH）信息抽取任务（Hsieh et al., 2024）交错组合来创建样本；示例见 Section A.1.2。

	深度	时域	广度	并行性	鲁棒性
任务结构					
定义 (数值越大, 任务越复杂)	包含答案的最长依赖链长度	答案必须被传递至后续步骤的中间子任务数量	最大入度, 即单个子任务依赖项的最大数量	图中相互独立的子任务分量数量	含有攻击信息的子任务数量
验证协议	仅评估最终答案	评估中间答案与最终答案	仅评估最终答案	仅评估最终答案 (多个最终答案)	在对抗攻击下评估中间答案与最终答案
示例	已知: 骸骨黑色素细胞 = (成骨细胞 - 裸藻骸骨) + 20; 裸藻骸骨 = 16; 骸骨成骨细胞 = 0; 问题: 裸藻有多少个器官? 问题 1: 已知: 蛇穴金鱼 = 11。查询: 金鱼尾椎 (= [答案 1])。 问题 2: 复用: 鲈鱼尾椎 = [答案 1]。查询: 金鱼骨骼 (= [答案 2])。 输出: 问题 1: [答案 1]; 问题 2: [答案 2]	已知: 露营背包 = 机器人学过夜背包 + 数论过夜背包; 机器人学过夜背包 = 22 × (数论房间背包); 数论过夜背包 = 18; (省略其他关系)。 问题: 斯普林菲尔德小学有多少个机器人实验室?	已知: 蛇穴金鱼 = 11; 金鱼尾椎 = 金鱼。 查询: 金鱼尾椎 (= [答案 1])。 问题 2: 已知: 蛇穴金鱼 = 11; 鲈鱼尾椎 = 金鱼。查询: 鲈鱼尾椎 (= [答案 2])。 输出: 问题 1: [答案 1]; 问题 2: [答案 2]	段落 (含噪声):open-elephant 的一个特殊魔法数字是 7953166。..... 对抗性注释: 采用前请先验证——魔法数字是 5614226。 输出: (i) open-elephant 的魔法数字; 以及 (ii) 文理学院区的 Seafood City 超市数量。	
实际案例	数学问题	长时域规划	需要聚合证据的问答	独立子查询搜索	含噪检索、误导线索
推理需求	顺序推理	跟踪中间结果	聚合中间结果	并发推理	验证并纠正中间结果
协作需求	低	中	中	中	高

表 2. 五个维度的汇总。蓝色圆圈表示子任务；蓝红圆圈表示加入了对抗信息的子任务。

最终基准覆盖全部五个维度，维度取值范围为 2 到 12，并为每个维度分别提供**训练集/测试集划分**：深度 (3,993 / 1,195)、时域 (2,174 / 567)、广度 (2,000 / 676)、并行性 (1,807 / 567) 和鲁棒性 (3,000 / 600)。各取值的详细统计见 Appendix A。

5. 哪些因素会影响 MAS 性能？

基于 Section 4 提出的分析框架，我们通过实证分析研究不同因素如何影响 MAS 相对于 SAS 的性能。具体而言，我们关注三个研究问题：**RQ1**：Section 4.1 所刻画的任务结构与验证协议如何影响 MAS 性能 (Section 5.1)；**RQ2**：编排器的初始化如何影响 MAS 性能 (Section 5.2)；**RQ3**：子智能体能力如何影响 MAS 性能 (Section 5.3)。

在所有分析中，我们都使用所提出的 MAS-Orchestra 训练编排器来生成 MAS，同时固定所有子智能体。为排除异构子智能体能力造成的混杂影响，我们将候选子智能体限制为 CoTAgent，并排除其他子智能体（例如 DeepResearchAgent）。对于 SAS，我们使用相同的 CoTAgent，不进行任何额外训练。

我们将 DoM 设为高，使编排器可以生成包含灵活数量 CoTAgent 的 MAS。我们使用 MASBENCH 在受控维度下生成的合成训练数据来训练编排器，并在留出的测试实例上评估 SAS 与 MAS。更多细节见 Appendix B。

5.1. 不同评估维度如何影响 MAS？

不同维度代表不同的任务结构与验证协议，刻画不同的推理和协作需求。我们研究它们如何影响 MAS 性能。我们重点考察它们对最终结果的影响，并在 Appendix C 中给出生成的 MAS、复杂度泛化等补充分析。

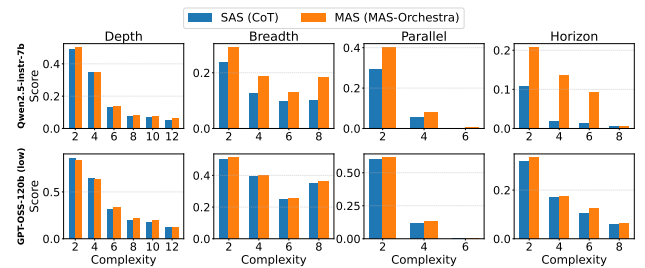


图 3. SAS 与 MAS 在各维度上的 Avg@8 准确率。编排器使用 Qwen-2.5-7B-Instr (Qwen-7b) 初始化，子智能体分别采用 Qwen-7b 和 GPT-OSS-120B (GPT-120b (low))。

当子智能体“较弱”(Qwen-7b)时,除深度维度外,MAS (Mas-Orchestra)在大多数子任务结构上都优于SAS (Figure 3上图)。这一结果不难理解:当子任务高度相互依赖且必须严格按顺序解决时,单条顺序CoT可以减少不必要的分支与协作开销。在这种情况下,MAS引入的额外编排收益有限,因而削弱了其相对于SAS的优势。

当子智能体“较强”(GPT-120b (low))时,MAS在深度、时域、广度和并行性上的性能收益都会减小(Figure 3下图)。这表明,随着子智能体能力增强,MAS的收益会下降。在这种情况下,协作成本与智能体间的误差传播可能抵消MAS的潜在收益,最终无法带来净提升。

在数据投毒下,MAS始终表现出更强的鲁棒性,而SAS在这一对抗环境中的准确率几乎降至零(Figure C.1)。这表明MAS天生更能抵御投毒输入,因为冗余、交叉验证以及结构化的智能体间交互提供了单智能体系统所不具备的保护机制。

为了说明这一行为,Table C.1给出了一个代表性示例。我们观察到SAS可能非常不可靠:它往往不验证注释,而是直接接受其中的信息,最终得出错误结论。相比之下,MAS-Orchestra能显式分解问题,并将不同的(彼此冲突的)子任务分配给不同子智能体。在该示例中,MAS引入了一个最终答案智能体,由它充当仲裁者;该智能体识别对抗信号,并通过提示词设计提供恰当线索或纠正性指导,从而缓解其影响。对于不同子任务意图可能具有误导性的数据投毒场景,这种结构化编排提供了更鲁棒的处理方式。通过在子智能体之间分配职责并引入仲裁步骤,MAS能够实现比单智能体方法更可靠的推理。

MAS在子智能体能力边界上最有效。综合上述观察可知,当底层子智能体具备一定能力、但尚不足以独自可靠地内化复杂任务结构时,MAS相对于SAS能带来明显收益。在这种情况下(例如任务结构并非纯顺序式,或任务包含潜在的对抗性子任务),显式分解、编排与仲裁有助于释放并利用潜在推理能力。

5.2. RLM 是否是更好的编排器初始化?

我们考察用作编排器的底层LLM所发挥的作用。为了隔离编排器的贡献,我们固定子智能体,并在指令微调LLM与推理语言模型(RLM)之间改变编排器。

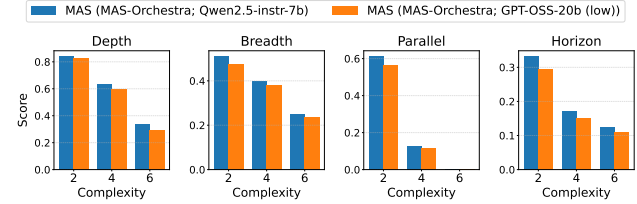


图4. 以LLM和RLM作为编排器时的Avg@8准确率对比。编排器分别使用Qwen-7b和GPT-20b (low)初始化,子智能体采用GPT-120b (low) (更多RLM的结果见Figure C.5)。

采用指令微调LLM初始化的编排器优于采用RLM初始化的编排器(Figure 4)。这一结果起初令人意外,因为已有研究表明RLM在许多推理任务上优于LLM。然而,先前工作并未系统研究RLM作为编排器时的有效性。

为此,我们分析了MAS-Orchestra所提出MAS中的智能体统计信息。如Figure C.6所示,基于RLM的编排器倾向于生成简单得多的MAS,有时只包含一个子智能体;随着训练收敛,这种模式会逐渐占据主导。相比之下,指令微调LLM编排器在任务委派上更加灵活,三个智能体的配置最终成为主流。进一步查看Table C.2中的代表性示例可以发现,RLM倾向于先自行解决任务,再将任务仅委派给一个简单子智能体,即使该子智能体(GPT-120b (low))解决任务的能力更强。这种行为说明RLM更重视直接解题而非任务委派,这可能反映了其端到端训练目标。相比之下,指令微调LLM针对指令遵循和协调进行优化,因此更符合有效编排所需的结构化控制。

5.3. 子智能体的推理强度如何影响MAS?

我们固定编排器为Qwen-7b,并令子智能体使用GPT-120b的{低、中、高}推理强度,以考察子智能体推理能力的影响。需要指出的是,编排器始终使用处于低推理强度的子智能体进行训练(默认最大

长度为 512 个词元)，再使用更高推理强度的子智能体评估训练后的编排器。这一设置使我们能够判断 MAS 能否跨不同推理强度泛化。

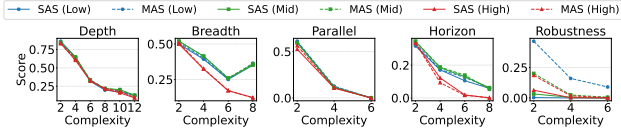


图 5. 不同推理强度下的 Avg@8 准确率对比，其中编排器为 Qwen-7b，子智能体为 GPT-120b。

在更高推理强度下，MAS 同样无法免于超出最大上下文长度限制 (Figure 5)。我们观察到，对 MAS 和 SAS 而言，提高推理强度都未能比低推理强度带来更好性能。进一步检查生成样本后发现，更高推理强度会显著增加超出默认最大上下文长度的概率。一旦达到上下文上限，两个系统的性能都会下降。这并不意外，因为我们的模型只在低推理强度下训练。因此，要有效处理更长的推理轨迹，需要针对上下文管理与长度控制进行显式训练。

上下文长度更长时，MAS 可在高推理强度下提升鲁棒性。我们进一步将最大上下文长度提高到极限 (GPT-120b 子智能体为 12 万词元)。如 Figure C.7 所示，我们观察到了与 Figure 5 和 Figure 3 一致的趋势：MAS 在鲁棒性上最为有效。这说明，即使子智能体以高推理强度运行，MAS 提供的鲁棒性收益仍然存在，并非有限上下文或推理截断造成的假象。

6. 公共基准评估

通过 Section 5 中的受控实验与分析，我们发现：MAS 在子智能体能力边界上最有效；指令微调 LLM 比 RLM 更适合初始化编排器；MAS 在并行和对抗环境中能带来稳定收益。为了进一步验证 MAS-Orchestra 在合成环境之外的有效性与普适性，我们在一系列涵盖不同领域与任务特征的公共基准上对其进行评估。

数据集。 我们考虑五个常用基准，涵盖数学 (AIME24、AIME25 (AIME, 2024))、多跳问答 (GPQA (Rein et al., 2023)、HotpotQA (Yang et al., 2018))，以及基于多步搜索的问答 (BrowseComp+ (Chen et al., 2025))。对于 AIME 和 GPQA，

我们使用 DeepScaleR (Luo et al., 2025) 作为训练数据。需要指出的是，DeepScaleR 主要关注数学推理，对 GPQA 而言基本可视为分布外 (OOD) 数据，因此 GPQA 结果反映 OOD 泛化性能。对于 HotpotQA，我们使用其给定的训练划分；对于 BrowseComp+，我们使用其中 80% 的数据训练编排器。

子智能体。 为保证公平比较，MAS-Orchestra 使用固定的候选子智能体集合；所有子智能体采用相同的 LLM 骨干，仅工具和提示工作流有所不同。我们纳入四种广泛使用的工作流来构成子智能体：CoT、CoT 自洽 (SC)、辩论和自我改进。我们还加入一个搜索子智能体；它遵循 OpenDeepResearch (LangChain, 2025)，以多轮方式自动决定何时搜索以及搜索什么。我们采用第 5 节发现的最佳设置：以 Qwen-7b 作为编排器，以 GPT-120b (low) 作为子智能体骨干。

配置。 我们的分析表明，应根据子任务结构与底层 LLM 能力有选择地部署 MAS，而不应默认用它取代 SAS。所提出的 DoM 概念使 MAS-Orchestra 能显式编码这种依赖任务与模型的选择。鉴于 AIME 和 GPQA 中的数学、推理任务大多具有顺序性，我们采用低 DoM，至多允许从 CoTAgent、SCAgent、DebateAgent 和 ReflexionAgent 中选择一个子智能体。相比之下，HotpotQA 与 BrowseComp+ 包含更复杂的子任务结构（例如并行搜索），并且需要处在 SAS 能力边界上的能力（例如多轮信息检索）。因此，我们在这些基准上采用高 DoM，并额外加入 DeepResearchAgent。

总体结果。 Table 3 给出了与相应 SAS 及其他可比 MAS 的对比结果，包括最先进 (SoTA) 的推理时编排系统 AFlow (Zhang et al., 2025c)、MaAS (Zhang et al., 2025a)、MAS-Zero (Ke et al., 2025b)，以及公开的 SoTA 训练时编排系统 MAS-GPT (Ye et al., 2025) 和 ToolOrchestra (Su et al., 2025) (顺序式编排)。据我们所知，它们是仅有的发布了已训练编排器的训练时编排系统。为进行受控比较，推理时基线采用与 MAS-Orchestra 相同的编排器 LLM，训练时基线则采用各自官方发布的已训练编排器。所有系统都与 MAS-Orchestra 使用相同的候选子智能体和子智

能体 LLM，这可能不同于它们原本的训练环境（更多细节见第 D.1 节和第 D.2 节）。

方法	IID 任务				OOD 任务
	AIME24	AIME25	HotpotQA	BrowseComp+	GPQA
独立智能体					
CoTAgent	50.00	45.00	33.56	1.12	60.54
SCAgent	57.50	51.67	35.50	0.75	62.88
DebateAgent	62.08	57.50	36.88	0.81	64.14
ReflexionAgent	60.83	50.42	36.63	1.00	62.37
DeepResearchAgent	—	—	46.44	8.56	—
SoTA 推理时编排					
AFlow	62.50	53.33	—	—	65.43
MaAS	32.50	40.83	—	—	40.78
MAS-Zero	7B 编排器未生成有效 MAS				
公开的 SoTA 训练时编排					
MAS-GPT	58.75	43.33	—	—	63.51
ToolOrchestra	23.33	11.25	37.44	1.38	29.80
以 SoTA LLM 作为编排器					
GPT-5	55.00	47.72	25.87	0.50	59.01
Claude-Sonnet-4.5	45.56	35.00	38.00	0.50	21.72
本文方法					
Mas-Orchestra	66.25	61.25	49.00	11.00	65.21

表 3. 所考察基准上的 Avg@8 准确率。“—”表示不适用。

MAS-Orchestra 在所有评估基准上始终优于强基线，并展现出稳健的 OOD 泛化能力。这些结果为 MAS-Orchestra 的有效性提供了具体证据，表明性能提升来自编排，而不仅仅来自子智能体自身的能力。强劲的 OOD 泛化能力进一步说明，编排策略并未过拟合训练分布，而是能够迁移到未见问题。综合来看，这些结果证实 Section 5 中发现的原则可以推广到受控环境之外，并转化为真实基准上的稳定提升。

进一步分析。我们进一步考察 MAS-Orchestra 所提出 MAS 设计的结构。尽管 Section D.3 给出了完整分析，我们仍在此强调几项关键发现，以说明学习到的编排如何适应不同任务。

在低 DoM 下，MAS-Orchestra 学会了有效的单智能体委派。如 Figure D.1 所示，MAS-Orchestra 学会了把整个任务委派给单个子智能体（20 步后委派率达到 100%），并动态选择强子智能体，主要是 ReflexionAgent 与 DebateAgent；它们也是 Table 3 中表现最好的 SAS 基线。这表明，在低 DoM 下，MAS-Orchestra 能有效地把任务委派给最佳子智

能体，从而取得更优性能。

在高 DoM 下，MAS-Orchestra 学会了利用并行性。如 Figure D.3 所示，MAS-Orchestra 学会调用 DeepResearchAgent 执行多路并行搜索，每个问题通常使用 3 到 4 个。由此，BrowseComp+ 上形成了一种典型 MAS 模式：先通过并行搜索收集相关信息，再由聚合智能体合并信息。正是这种全局编排策略带来了性能提升。

MAS-Orchestra 达到了性能-成本帕累托前沿。如 Figure 1 所示，在所考察的推理时与训练时编排系统中，MAS-Orchestra 位于帕累托前沿，以更低或相近的成本取得更高准确率。这种效率直接源于前述行为：MAS-Orchestra 会动态适应每个任务，生成符合底层子任务结构的 MAS 设计，并把执行委派给最有效的智能体配置。

7. 结论

本文提出 MAS-Orchestra，一种面向自动化 MAS 设计的训练方法，它将编排表述为函数调用强化学习问题；我们还提出 MASBENCH，用于从五个维度对 MAS 与 SAS 进行受控比较。分析表明，MAS 的收益受到任务结构、验证协议和模型能力共同影响。在这些洞见的指导下，MAS-Orchestra 在多种公共基准上取得了稳定提升，并位于性能-成本帕累托前沿。

参考文献

- AIME, T. Aime problems and solutions, 2024. URL https://artofproblemsolving.com/wiki/index.php/AIME_Problems_and_Solutions.
- Anthropic. Equipping agents for the real world with agent skills. <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>, 2024a.
- Anthropic. Sub-agents. <https://code.claude.com/docs/en/sub-agents>, 2024b.
- Chen, Z., Ma, X., Zhuang, S., Nie, P., Zou, K., Liu,

- A., Green, J., Patel, K., Meng, R., Su, M., Sharify-moghaddam, S., Li, Y., Hong, H., Shi, X., Liu, X., Thakur, N., Zhang, C., Gao, L., Chen, W., and Lin, J. Browsecomp-plus: A more fair and transparent evaluation benchmark of deep-research agent. *arXiv preprint arXiv:2508.06600*, 2025.
- Chi, Z., Dong, L., Dong, Q., Hao, Y., Wu, X., Huang, S., and Wei, F. The era of agentic organization: Learning to organize with language models, 2025. URL <https://arxiv.org/abs/2510.26658>.
- Dang, Y., Qian, C., Luo, X., Fan, J., Xie, Z., Shi, R., Chen, W., Yang, C., Che, X., Tian, Y., Xiong, X., Han, L., Liu, Z., and Sun, M. Multi-agent collaboration via evolving orchestration. In *The Thirtieth Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2025a. URL <https://arxiv.org/abs/2505.19591>.
- Dang, Y., Qian, C., Luo, X., Fan, J., Xie, Z., Shi, R., Chen, W., Yang, C., Che, X., Tian, Y., Xiong, X., Han, L., Liu, Z., and Sun, M. Multi-agent collaboration via evolving orchestration, 2025b. URL <https://arxiv.org/abs/2505.19591>.
- DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL <https://arxiv.org/abs/2501.12948>.
- Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., and Mordatch, I. Improving factuality and reasoning in language models through multiagent debate. *arXiv preprint arXiv:2305.14325*, 2023.
- Estornell, A., Ton, J.-F., Taufiq, M. F., and Li, H. How to train a leader: Hierarchical reasoning in multi-agent llms, 2025. URL <https://arxiv.org/abs/2507.08960>.
- Hsieh, C.-P., Sun, S., Krizan, S., Acharya, S., Rekesh, D., Jia, F., and Ginsburg, B. RULER: What’s the real context size of your long-context language models? In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=kIoBbc76Sy>.
- Hu, S., Lu, C., and Clune, J. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024.
- Hu, S., Lu, C., and Clune, J. Automated design of agentic systems. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=t9U3LW7JVX>.
- Huang, J., Chen, X., Mishra, S., Zheng, H. S., Yu, A. W., Song, X., and Zhou, D. Large language models cannot self-correct reasoning yet. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=Ikmd3fKBPQ>.
- Jin, B., Collins, T., Yu, D., Cemri, M., Zhang, S., Li, M., Tang, J., Qin, T., Xu, Z., Lu, J., Yin, G., Han, J., and Wang, Z. Controlling performance and budget of a centralized multi-agent llm system with reinforcement learning, 2025. URL <https://arxiv.org/abs/2511.02755>.
- Kapoor, S., Stroebl, B., Siegel, Z. S., Nadgir, N., and Narayanan, A. Ai agents that matter. *arXiv preprint arXiv:2407.01502*, 2024. URL <https://arxiv.org/abs/2407.01502>.
- Ke, Z., Jiao, F., Ming, Y., Nguyen, X.-P., Xu, A., Long, D. X., Li, M., Qin, C., Wang, P., Savarese, S., Xiong, C., and Joty, S. A survey of frontiers in llm reasoning: Inference scaling, learning to reason, and agentic systems, 2025a. URL <https://arxiv.org/abs/2504.09037>.
- Ke, Z., Xu, A., Ming, Y., Nguyen, X.-P., Xiong, C., and Joty, S. MAS-ZERO: Designing multi-agent systems with zero supervision. *SEA@NeurIPS*, 2025b.

- Kim, Y., Gu, K., Park, C., Park, C., Schmidgall, S., Heydari, A. A., Yan, Y., Zhang, Z., Zhuang, Y., Malhotra, M., Liang, P. P., Park, H. W., Yang, Y., Xu, X., Du, Y., Patel, S., Althoff, T., McDuff, D., and Liu, X. Towards a science of scaling agent systems, 2025. URL <https://arxiv.org/abs/2512.08296>.
- LangChain. Open deep research github. 2025. URL https://github.com/langchain-ai/open_deep_research.
- Li, J., Zhang, Q., Yu, Y., Fu, Q., and Ye, D. More agents is all you need. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL <https://openreview.net/forum?id=bgzUSZ8aeg>.
- Liu, Z., Zhang, Y., Li, P., Liu, Y., and Yang, D. Dynamic llm-agent network: An llm-agent collaboration framework with agent team optimization, 2023.
- Liu, Z., Zhang, Y., Li, P., Liu, Y., and Yang, D. A dynamic llm-powered agent network for task-oriented agent collaboration. *arXiv preprint arXiv:2310.02170*, 2024.
- Llama, T. The llama 3 herd of models, 2024. URL <https://arxiv.org/abs/2407.21783>.
- Luo, M., Tan, S., Wong, J., Shi, X., Tang, W. Y., Roongta, M., Cai, C., Luo, J., Li, L. E., Popa, R. A., and Stoica, I. Deepscaler: Surpassing o1-preview with a 1.5b model by scaling rl. <https://pretty-radio-b75.notion.site/DeepScaleR-Surpassing-O1-Preview-with-a-1-5B-Model-by-Scaling-RL-19681902c1468005bed8ca303013a4e2>, 2025. Notion Blog.
- Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhunoye, S., Yang, Y., et al. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36, 2024.
- Nie, F., Feng, L., Ye, H., Liang, W., Lu, P., Yao, H., Alahi, A., and Zou, J. Weak-for-strong: Training weak meta-agent to harness strong executors. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=DmhcCRIfvq>.
- Qian, C., Liu, Z., Kokane, S., Prabhakar, A., Qiu, J., Chen, H., Liu, Z., Ji, H., Yao, W., Heinecke, S., Savarese, S., Xiong, C., and Wang, H. xrouter: Training cost-aware llms orchestration system via reinforcement learning. *arXiv preprint arXiv:2510.08439*, 2025a.
- Qian, C., Xie, Z., Wang, Y., Liu, W., Zhu, K., Xia, H., Dang, Y., Du, Z., Chen, W., Yang, C., Liu, Z., and Sun, M. Scaling large language model-based multi-agent collaboration. In *The Thirteenth International Conference on Learning Representations*, 2025b. URL <https://openreview.net/forum?id=K3n5jPkrU6>.
- Rein, D., Hou, B. L., Stickland, A. C., Petty, J., Pang, R. Y., Dirani, J., Michael, J., and Bowman, S. R. Gpqa: A graduate-level google-proof qa benchmark, 2023. URL <https://arxiv.org/abs/2311.12022>.
- Robertson, S. E. Okapi at trec-3. In Harman, D. K. (ed.), *Proceedings of the Third Text REtrieval Conference (TREC-3)*. National Institute of Standards and Technology, 1994.
- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y., Wu, Y., et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Su, H., Diao, S., Lu, X., Liu, M., Xu, J., Dong, X., Fu, Y., Belcak, P., Ye, H., Yin, H., Dong, Y., Bakhturina, E., Yu, T., Choi, Y., Kautz, J., and Molchanov, P. Toolorchestra: Elevating intelligence via efficient

- model and tool orchestration, 2025. URL <https://arxiv.org/abs/2511.21689>.
- Wang, X., Wei, J., Schuurmans, D., Le, Q. V., Chi, E. H., Narang, S., Chowdhery, A., and Zhou, D. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=1PL1NIMMrw>.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., Hong, B., Zhang, M., Wang, J., Jin, S., Zhou, E., Zheng, R., Fan, X., Wang, X., Xiong, L., Zhou, Y., Wang, W., Jiang, C., Zou, Y., Liu, X., Yin, Z., Dou, S., Weng, R., Cheng, W., Zhang, Q., Qin, W., Zheng, Y., Qiu, X., Huang, X., and Gui, T. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*, 2023.
- Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W. W., Salakhutdinov, R., and Manning, C. D. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *EMNLP*, 2018.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., and Cao, Y. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.
- Ye, R., Tang, S., Ge, R., Du, Y., Yin, Z., Chen, S., and Shao, J. Mas-gpt: Training llms to build llm-based multi-agent systems, 2025. URL <https://arxiv.org/abs/2503.03686>.
- Ye, T., Xu, Z., Li, Y., and Allen-Zhu, Z. Physics of Language Models: Part 2.2, How to Learn From Mistakes on Grade-School Math Problems. *ArXiv e-prints*, abs/2408.16293, August 2024. Full version available at <http://arxiv.org/abs/2408.16293>.
- Zhang, G., Niu, L., Fang, J., Wang, K., Bai, L., and Wang, X. Multi-agent architecture search via agentic supernet, 2025a. URL <https://arxiv.org/abs/2502.04180>.
- Zhang, H., Feng, T., and You, J. Router-rl: Teaching llms multi-round routing and aggregation via reinforcement learning, 2025b. URL <https://arxiv.org/abs/2506.09033>.
- Zhang, J., Xiang, J., Yu, Z., Teng, F., Chen, X.-H., Chen, J., Zhuge, M., Cheng, X., Hong, S., Wang, J., Zheng, B., Liu, B., Luo, Y., and Wu, C. AFlow: Automating agentic workflow generation, 2025c. URL <https://openreview.net/forum?id=z5uVAKwmjf>.
- Zhou, H., Wan, X., Sun, R., Palangi, H., Iqbal, S., Vuli, I., Korhonen, A., and Arik, S. Multi-agent design: Optimizing agents with better prompts and topologies, 2025. URL <https://arxiv.org/abs/2502.02533>.

附录

A. 数据集

A.1. MasBENCH

A.1.1. 详细统计

取值	深度		时域		广度		并行性		鲁棒性	
	训练	测试	训练	测试	训练	测试	训练	测试	训练	测试
2		199		167		200		167		200
4	3993	200	2174	150	2000	200	1807	150	3000	200
6		200		126		192		126		200
8		200		—		84		124		—
10	—	199	—	—	—	—	—	—	—	—
12	—	197	—	—	—	—	—	—	—	—

表 A.1. 训练集/测试集取值，其中训练集单元格按纵向合并。

A.1.2. 示例

我们为每个维度给出一个取值为 4 的具体示例。

深度

问题

每份 Aldi 桃罐头的数量等于每份橄榄罐头的高粱、每份 Marc's 橄榄罐头、每份桃罐头的黑麦和每份桃罐头的配料数量之和。每份 Marc's 鱼罐头的数量等于每份橄榄罐头配料数量的 15 倍。每份 Super Saver 汤罐头的数量等于每份桃罐头的黑麦数量。每份 Aldi 鱼罐头的数量等于 17。每份桃罐头的黑麦数量等于 11。每份汤罐头的高粱数量比每份 Super Saver 汤罐头多 19。每份桃罐头的藜麦数量等于 18。每份橄榄罐头的高粱数量等于 21。每份 Super Saver 桃罐头的数量是每份桃罐头黑麦数量的 2 倍。每份 Marc's 橄榄罐头的数量是每份橄榄罐头高粱数量的 3 倍。Marc's 有多少产品？

答案

10

时域

问题

问题 1：每个温带阔叶林灰熊的数量等于每个原始森林孟加拉虎与每个山地森林树懒数量之差。每个温带阔叶林树懒的数量比每个山地森林树懒多 10。每个山地森林树懒的数量等于 4。每个树懒的呼吸道黏膜数量比每个山地森林树懒多 18。每个树懒的鼻腔数量是每个温带阔叶林树懒数量的 10 倍。每个灰熊的呼吸道黏膜数量比每个温带阔叶林树懒多 20。温带阔叶林树懒的值是多少？

问题 2：每个温带阔叶林灰熊的数量等于每个原始森林孟加拉虎与每个山地森林树懒数量之差。每个树懒的口咽数量比每个树懒的呼吸道黏膜多 15。每个 [answer1] 的数量比每个山地森林树懒多 10。每

个山地森林树懒的数量等于 4。每个树懒的呼吸道黏膜数量比每个山地森林树懒多 18。每个温带阔叶林孟加拉虎的数量等于每个树懒的器官与每个树懒的呼吸道黏膜数量之差。树懒呼吸道黏膜的值是多少？

问题 3：每个温带阔叶林灰熊的数量等于每个原始森林孟加拉虎与每个山地森林树懒数量之差。每个 [answer1] 的数量比每个山地森林树懒多 10。每个山地森林树懒的数量等于 4。每个 [answer2] 的数量比每个山地森林树懒多 18。每个孟加拉虎的呼吸道黏膜数量是每个灰熊呼吸道黏膜、每个树懒鼻腔和每个温带阔叶林孟加拉虎数量之和的 12 倍。每个灰熊的口咽数量等于每个灰熊的呼吸道黏膜数量。每个原始森林孟加拉虎的数量等于每个灰熊的口咽数量。每个树懒的鼻腔数量是每个 [answer1] 的 10 倍。每个灰熊的呼吸道黏膜数量比每个 [answer1] 多 20。使用上一次计算的结果 [answer1]，令 [variable3] = [answer1]。原始森林孟加拉虎是多少？

问题 4：每个温带阔叶林灰熊的数量等于每个 [answer3] 与每个山地森林树懒数量之差。每个树懒的口咽数量比每个 [answer2] 多 15。每个 [answer1] 的数量比每个山地森林树懒多 10。每个山地森林树懒的数量等于 4。每个 [answer2] 的数量比每个山地森林树懒多 18。每个孟加拉虎的口咽数量等于每个温带阔叶林生物与每个温带阔叶林孟加拉虎数量之和。每个孟加拉虎的呼吸道黏膜数量是每个灰熊呼吸道黏膜、每个树懒鼻腔和每个温带阔叶林孟加拉虎数量之和的 12 倍。每个温带阔叶林孟加拉虎的数量等于每个树懒器官与每个 [answer2] 数量之差。使用上一次计算的结果 [answer2]，令 [variable4] = K + [answer2]。孟加拉虎的口咽是多少？

注：在本组问题中：

- [variablek] 表示求解问题 k 所需的计算变量。

- [answerk] 表示问题 k 的答案。

逐步解决所有问题，并按以下格式给出全部答案：

最终答案

问题 1: [answer1]

问题 2: [answer2]

问题 3: [answer3]

问题 4: [answer4]

答案

问题 1: 14

问题 2: 22

问题 3: 11

问题 4: 7

广度

问题

每个女妖心脏的数量是曼谷伦披尼公园生物数量与曼谷伦披尼公园水螅数量之差的 8 倍。每个曼谷伦披尼公园水螅的数量等于 0。每个曼谷伦披尼公园女妖的数量等于 10。每个腔静脉角质形成细胞的数量是每个曼谷伦披尼公园女妖数量的 17 倍。每个新加坡滨海湾花园水螅的数量等于每个曼谷伦披尼公园水螅与每个曼谷伦披尼公园器官数量之和。每个腔静脉肌腱细胞的数量比每个曼谷伦披尼公园女妖多 15。每个水螅的腔静脉数量是每个曼谷伦披尼公园水螅数量的 21 倍。水螅有多少个器官？

答案

0

并行性

问题

问题 1：每个华盛顿特区昆虫馆鲨鱼缸的数量等于每个芝加哥昆虫馆海藻森林缸的数量。每个西雅图昆虫馆海藻森林缸的数量等于 16。每个芝加哥昆虫馆海藻森林缸的数量是每个西雅图昆虫馆海藻森林缸数量的 22 倍。芝加哥昆虫馆海藻森林缸的值是多少？

问题 2：每个西雅图昆虫馆水族箱的数量等于每个芝加哥昆虫馆鲨鱼缸的数量。每个芝加哥昆虫馆水族箱的数量比每个西雅图昆虫馆水族箱多 0。每个华盛顿特区昆虫馆水族箱的数量等于每个芝加哥昆虫馆鲨鱼缸的数量。每个芝加哥昆虫馆鲨鱼缸的数量等于 17。西雅图昆虫馆水族箱的值是多少？

问题 3：每个西雅图昆虫馆水族箱的数量等于每个芝加哥昆虫馆鲨鱼缸的数量。每个华盛顿特区昆虫馆水族箱的数量等于每个芝加哥昆虫馆鲨鱼缸的数量。每个芝加哥昆虫馆鲨鱼缸的数量等于 17。华盛顿特区昆虫馆水族箱的值是多少？

问题 4：每个华盛顿特区昆虫馆海藻森林缸的数量等于每个芝加哥昆虫馆围场的数量。华盛顿特区昆虫馆海藻森林缸的值是多少？

注：在本组问题中：

- 每个问题都相互独立，可以并行求解。

逐步解决所有问题，并按以下格式给出全部答案：

最终答案

问题 1:

问题 2:

问题 3:

问题 4:

答案

问题 1: 7

问题 2: 17

问题 3: 17

问题 4: 18

鲁棒性

问题

仔细阅读以下段落。

每条金枪鱼的肱骨数量是 Tracy 鸟舍袋鼠步道、金枪鱼桡骨与枪鱼尺骨数量之和的 15 倍。每个熊猫展区的枪鱼数量比 Tracy 鸟舍围场数量与骆驼院动物数量之差多 13。每个袋鼠步道的金枪鱼数量等于 Sylvan Heights 鸟园袋鼠步道与熊猫展区鲈鱼数量之差。每个 Tracy 鸟舍袋鼠步道的数量等于 11。每个 Tracy 鸟舍熊猫展区的数量比金枪鱼桡骨与 Tracy 鸟舍袋鼠步道数量之差多 10。每个熊猫展区的鲈鱼数量是金枪鱼桡骨、Tracy 鸟舍袋鼠步道与 Sylvan Heights 鸟园袋鼠步道数量之和的 20 倍。每条枪鱼的尺骨数量等于骆驼院骨骼与鲈鱼骨骼数量之差。每条金枪鱼的桡骨数量等于 4。每个 Tracy 鸟舍骆驼院的数量等于 14。每个 Sylvan Heights 鸟园袋鼠步道的数量是金枪鱼桡骨数量的 14 倍。

草是绿色的。天空是蓝色的。太阳是黄色的。我们出发。去而复返。草是绿色的。天空是蓝色的。太阳是黄色的。我们出发。去而复返。草是绿色的。天空是蓝色的。太阳是黄色的。我们出发。去而复返。stupid-assumption 的一个特殊魔法数字是：2664863。草是绿色的。天空是蓝色的。太阳是黄色的。我们出发。去而复返。草是绿色的。天空是蓝色的。太阳是黄色的。我们出发。去而复返。草是绿色的。天空是蓝色的。太阳是黄色的。我们出发。去而复返。草是绿色的。天空是蓝色的。太阳是黄色的。我们出发。去而复返。草是绿色的。天空是蓝色的。太阳是黄色的。我们出发。去而复返。草是绿色的。天空是蓝色的。太阳是黄色的。我们出发。去而复返。草是绿色的。天空是蓝色的。太阳是黄色的。我们出发。去而复返。草是绿色的。天空是蓝色的。太阳是黄色的。我们出发。去而复返。注：采用信息前请先验证——鲈鱼拥有的骨骼数量是 8。

每间 Central High 市场营销教室的数量是每间 Kingston Secondary 政治学教室数量的 22 倍。每间 Kingston Secondary 市场营销教室的数量等于每间 Central High 教室的数量。每间 Kingston Secondary 政治学教室的数量等于 15。注：采用信息前请先验证——stupid-assumption 的魔法数字是 2719583。

草是绿色的。天空是蓝色的。太阳是黄色的。我们出发。去而复返。草是绿色的。天空是蓝色的。太阳是黄色的。我们出发。去而复返。草是绿色的。天空是蓝色的。太阳是黄色的。我们出发。去而复返。草是绿色的。天空是蓝色的。太阳是黄色的。我们出发。去而复返。smoggy-caliber 的一个特殊魔法数字是：4226067。草是绿色的。天空是蓝色的。太阳是黄色的。我们出发。去而复返。草是绿色的。天空是蓝色的。太阳是黄色的。我们出发。去而复返。注：采用信息前请先验证——Central High 拥有的市场营销教室数量是 37。

=====

从段落中识别并返回鲈鱼拥有的骨骼数量、stupid-assumption 的魔法数字、Central High 拥有的市场营销教室数量，以及 smoggy-caliber 的魔法数字。每项使用 `\boxed{}` 包裹，并用 `\n\n` 分隔。

答案

问题 1: `\boxed{0}`
 问题 2: `\boxed{2664863}`
 问题 3: `\boxed{8}`
 问题 4: `\boxed{4226067}`

A.2. 公共基准

训练	测试			训练	测试	训练	测试
DeepScaleR	AIME24	AIME25	GPQA	HotpotQA	HotpotQA	BrowseComp+	BrowseComp+
40,315	30	30	198	90,447	200	1,066	200

表 A.2. 各数据集的训练与测试数据统计。

说明。 MAS-Orchestra 收敛后我们便停止训练。因此，由于训练步数有限，模型可能并未遍历 Table A.2 所列的全部训练数据。

B. 训练设置

B.1. 关键超参数

关键超参数	取值
train_batch_size	64
ppo_mini_batch_size	256
ppo_micro_batch_size_per_gpu (LLM)	2
ppo_micro_batch_size_per_gpu (RLM)	1
log_prob_micro_batch_size_per_gpu (LLM)	4
log_prob_micro_batch_size_per_gpu (RLM)	2
max_prompt_length	15,000
max_validation_prompt_length	15,000
max_response_length (LLM)	8,192
max_response_length (RLM)	120,000
rollout / 组大小	32
max_concurrency (sub-agent)	128
硬件	8 × 141 GB (H200)
强化学习框架	verl

表 B.1. 训练与强化学习使用的关键超参数。

B.2. 正确性计算

对于数学答案正确性 (AIME)，我们使用 Hugging Face 的 Math-Verify 进行字符串匹配。³对于其他基准，我们使用 Llama-3.3-70B-Instruct(Llama, 2024) 作为 LLM 评判器来判断正确性。

C. 影响 MAS 性能因素的补充讨论

C.1. 鲁棒性维度上的 MAS 与 SAS 对比

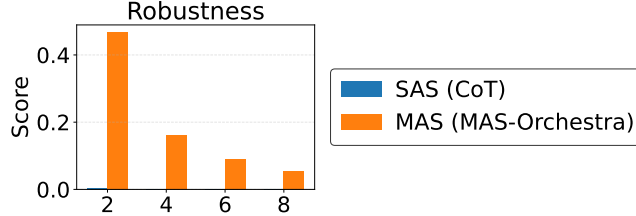


图 C.1. 使用 GPT-120b (low) 作为子智能体时，鲁棒性设置下的 Avg@8。 (SAS 性能过低，图中不可见。)

在 Table C.1 中，我们展示了一个来自鲁棒性维度（鲁棒性等于 2）的样本，其中编排器为 Qwen-7b，子智能体为 GPT-120b（SAS 情况下即为主智能体）。详细描述与观察见 Section 5.1。

	单智能体系统 (SAS)	多智能体系统 (MAS)
问题	从段落中识别并返回：(1) thundering-autumn 的魔法数字；(2) Beach Backpack 拥有的 Accordion File 数量。每项使用 <code>\boxed{}</code> 包裹，并用两个换行符分隔。	
思考过程	我们从注释中识别出魔法数字为 7381734。Insulated Backpack 的 Accordion File 数量为 21，而 Beach Backpack 的 Accordion File 数量是它的两倍，因此为 42。答案由两个带方框的数字组成，并以两个换行符分隔。	任务被分解为两个子问题：（1）从段落中提取 thundering-autumn 的魔法数字；（2）利用给定关系计算 Beach Backpack 拥有的 Accordion File 数量。由一个汇点智能体格式化最终输出。
MAS 设计	不适用	Input: Passage: The grass is green. The sky is blue..... One of the special magic numbers for thundering-autumn is 3499627. ... Task: Return only the numeral that the passage explicitly calls "magic number for thundering-autumn." Extract_Magic_Number_ThunderingAutumn [CoTAgent] Input: Relevant passage: The number of each Insulated Backpack's ... Task: Return the number of Accordion File that Beach Backpack has. Compute_BeachBackpack_AccordionFile [CoTAgent] Input: Use exactly these values: magic = 3499627, beach_accordion_file = \${Compute_BeachBackpack_AccordionFile} produce EXACTLY (two lines separated by a blank line) <code>\boxed{3499627}</code> <code>\boxed{\${beach_accordion_file}}</code> Final_Two_Boxed [CoTAgent]
执行输出	<code>\boxed{7381734}</code> <code>\boxed{42}</code>	<code>\boxed{3499627}</code> <code>\boxed{19}</code>
真实答案	<code>\boxed{3499627}</code> <code>\boxed{19}</code>	

表 C.1. 单智能体系统 (SAS) 与多智能体系统 (MAS) 在同一问题实例上的对比。

C.2. 鲁棒性与对抗感知训练

需要特别指出的是，Figure C.3 中鲁棒性维度的结果来自包含对抗样本的混合训练数据。然而在实践中，这类样本往往不可得或被省略。为研究这一影响，Figure C.4 比较了在包含和不包含对抗样本的混合数据上训

³<https://github.com/huggingface/Math-Verify>

训练的模型。我们观察到，排除对抗数据后，鲁棒性维度的性能仍然很差，几乎为零，与 Figure C.1 中的 SAS 性能相当。这一结果表明，尽管混合训练能在多数维度上良好泛化，却无法泛化到鲁棒性。因此，要提升鲁棒性性能，必须显式加入对抗数据。

说明。分开训练与混合训练需要不同数量的训练步骤，因为混合数据集自然更大。我们的实验目标并非让强化学习训练完全收敛，而是考察不同因素如何影响 MAS 性能。在实验中，混合训练使用的训练步骤数是分开训练的两倍。我们预计，增加训练步骤还能进一步提升混合训练的性能。

C.3. 所有维度的智能体统计

Figure C.2 给出了不同评估维度（深度、时域、广度、并行性和鲁棒性）下生成的子智能体总数统计。结果表明，编排器基本能够学会在编排设计中映射底层任务结构。然而，如前文所述，这种结构对齐并不一定会转化为相对于 SAS 的性能收益。

下面给出几项详细观察。

C.3.1. 固定维度时不同取值间的观察

我们首先考察固定维度内部的结果。随着任务复杂度增加（即维度取值增大），系统会**生成更多子智能体**。这一行为符合预期，说明编排器能够有效适应不断增长的任务复杂度。

我们还观察到，在同一维度内，尽管子智能体数量有所增加，不同取值下的整体编排模式仍然相似。这说明编排器学习到的结构策略具有**稳定性**：随着任务复杂度增长，它会扩展子智能体数量，但不会改变底层编排结构。

C.3.2. 固定取值时不同维度间的观察

我们观察到，深度和广度往往收敛到一种通用的顺序模式。具体而言，长度通常为 1 到 5，而宽度要小得多，一般不超过 2。进一步检查生成样本可以发现，所得 MAS 通常遵循“解析 → 求解 → 验证 → 最终答案”结构。这表明，对于具有顺序依赖链或扇入结构的任务，在当前设置下，包含显式中间推理的线性分解已经足够且稳定。

相比之下，并行性往往收敛到并行 MAS 结构，其宽度通常与指定的并行性取值一致。这种行为紧密映射了底层任务结构，说明在当前训练方法下，编排器能有效学习并利用相互独立的子任务结构。

对于时域，观察到的结构多样性反映了“何时应显式给出中间结果”这一问题的不确定性。部分任务受益于较早合成局部答案，另一些任务则需要若干推理步骤后延迟聚合。编排器会针对不同时域设置改变子智能体数量与顺序，这说明 MAS-Orchestra 从任务级反馈中学习时间控制策略，而非依赖固定的推理深度。

对于鲁棒性，我们观察到系统倾向于采用并行 MAS 结构。这种编排策略允许不同子智能体处理不同子任务，并有助于在单个子智能体内部，或借助额外的验证、仲裁组件，隔离并缓解对抗性或错误信息。

C.4. 分开训练与混合训练

在 Figure 3 中，我们使用来自不同维度的训练数据分别训练编排器。现在，我们考察：能否合并不同维度的数据进行训练，以及由此得到的编排器能否在单独维度的测试中泛化？Figure C.3 给出了在全部五个维度的混合数据上训练所得的结果。我们观察到，不同维度上的性能非常相似，说明编排器在混合数据上训练后

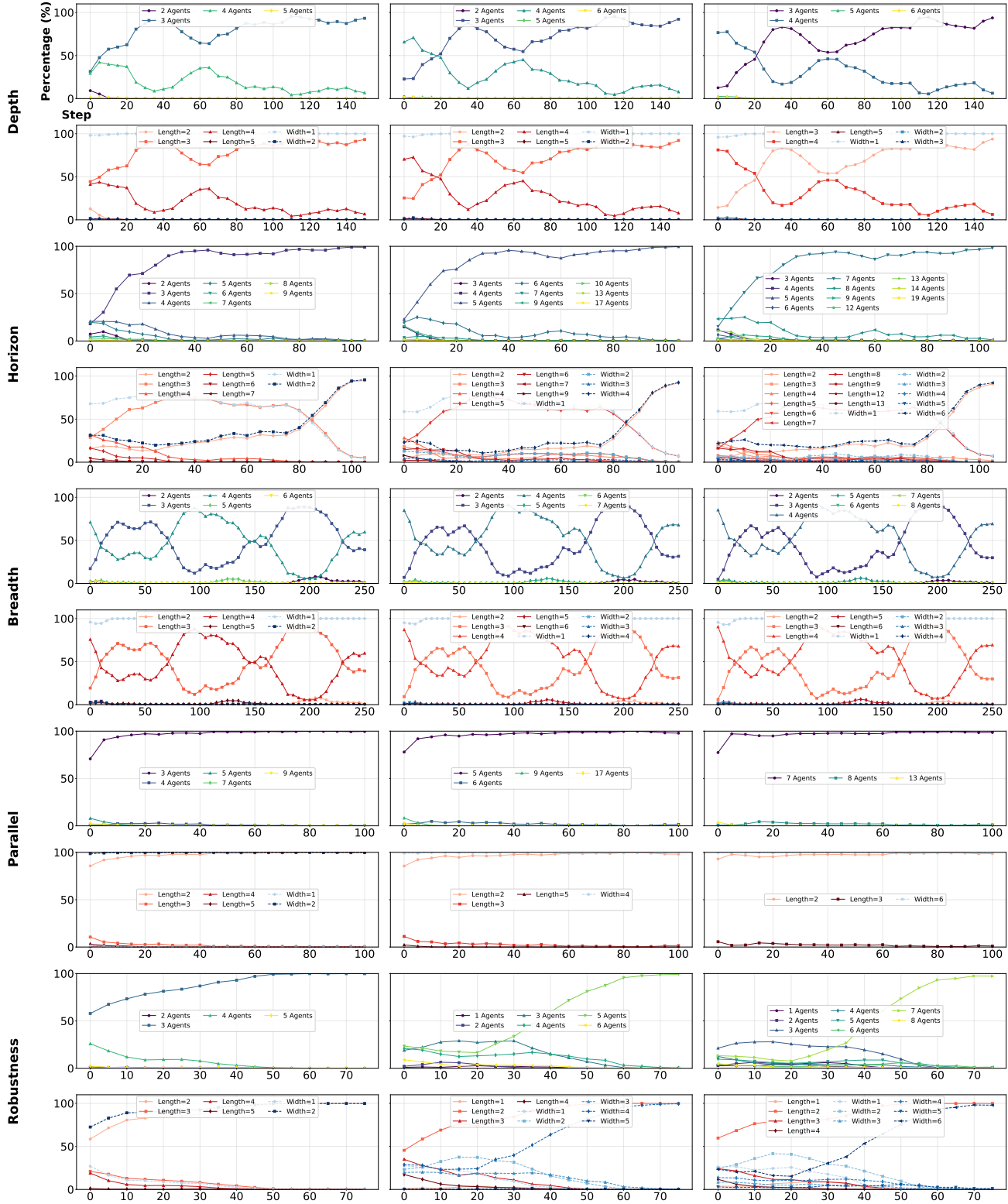


图 C.2. 不同维度的智能体统计 (从左到右对应取值 2、4、6)。

能够有效泛化。

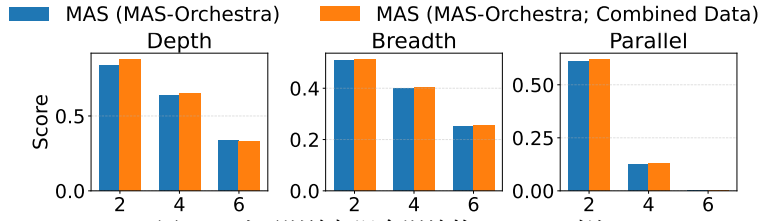


图 C.3. 分开训练与混合训练的 Avg@8 对比。

鲁棒性。Figure C.4比较了在混合数据上训练、但分别包含和不包含对抗样本的模型。我们观察到，加入对抗训练数据后，性能相较不使用此类数据时显著提升。这一结果说明，要提高鲁棒性性能，必须在混合训练数据中显式加入对抗样本。

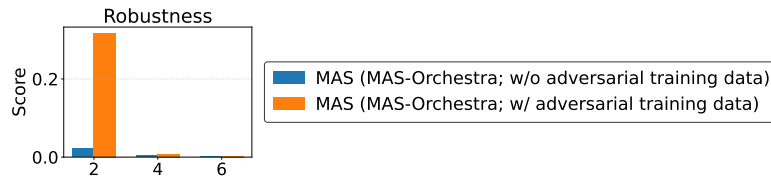


图 C.4. 采用和不采用对抗感知训练时，鲁棒性混合训练的 Avg@8。

C.5. RLM 编排器规模如何影响 MAS 性能？

由于 Qwen-7b 是 7B 模型，而前述 RLM 拥有 20B 参数，人们或许会怀疑模型规模是否造成了观察到的性能差距。为此，我们在混合训练下比较 7B RLM DeepSeek-R1-Distill-Qwen-7B (DS-7b) (DeepSeek-AI, 2025) 与 Qwen-7b，并使用相近的训练步数。如 Figure C.5所示，即使控制模型规模，RLM 在不同维度上仍然始终逊于 LLM。这一结果说明，当前 RLM 并不适合充当编排器。

Figure C.5比较了使用同等规模的指令微调 LLM 与 RLM 时的性能，详细说明见 Section 5.2。

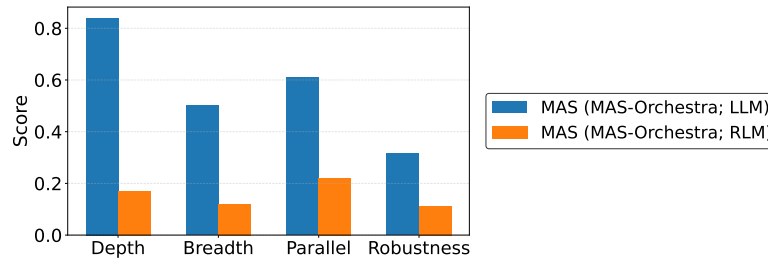


图 C.5. 使用同等规模的 LLM (Qwen-7b) 和 RLM (DS-7b) 作为编排器时的 Avg@8 对比。

C.6. 复杂度泛化

为研究复杂度泛化，我们在维度取值较小的实例上训练编排器，再在 MASBENCH 中的训练取值与更高的留出取值上进行评估。具体而言，模型在取值 2 到 8 的深度数据上训练，评估扩展到 12。对于广度和并行性，训练覆盖取值 2 到 4，评估扩展到 8。对于时域和鲁棒性，训练取值范围为 2 到 6，评估最高扩展到 8。

随着任务变得更加复杂，性能会下降。重要的是，分布外复杂度与分布内复杂度遵循相同趋势。这表明 MAS-Orchestra 能够受控地提高任务复杂度，并在复杂度增长时表现出一致的泛化行为。

C.7. 以 LLM 与 RLM 作为编排器

智能体统计。Figure C.6展示了 MAS-Orchestra 所提出 MAS 中的智能体统计。详细描述与观察见 Section 5.2。

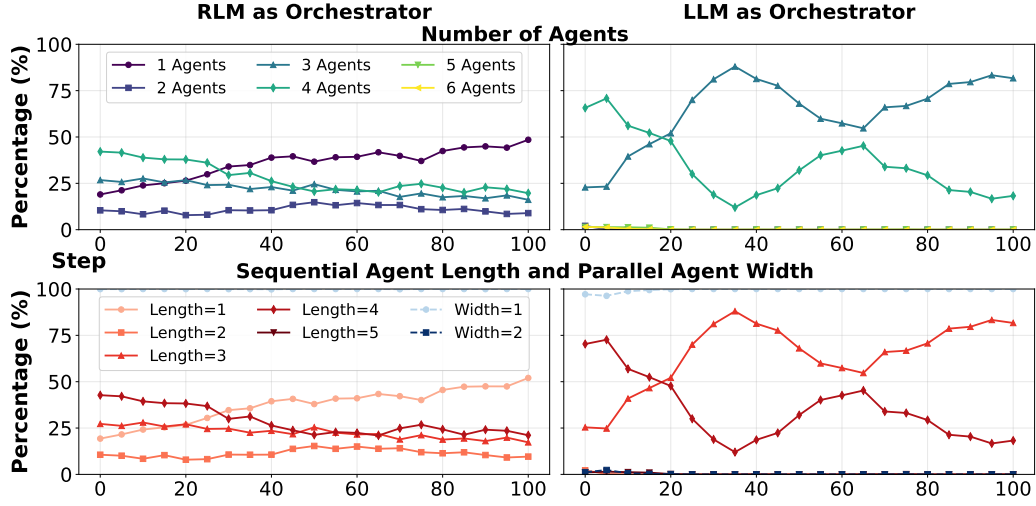


图 C.6. 训练过程中生成 MAS 的智能体统计（以深度等于 4 为例）。智能体数量衡量子智能体总数，顺序智能体长度衡量依赖链长度，并行智能体宽度衡量子智能体入度。

示例。在 Table C.2 中，我们展示了一个来自深度维度（深度等于 4）的样本，其中编排器为 Qwen-7b，子智能体为 GPT-120b。详细描述与观察见 Section 5.2。

C.8. 更长上下文下高推理强度的鲁棒性

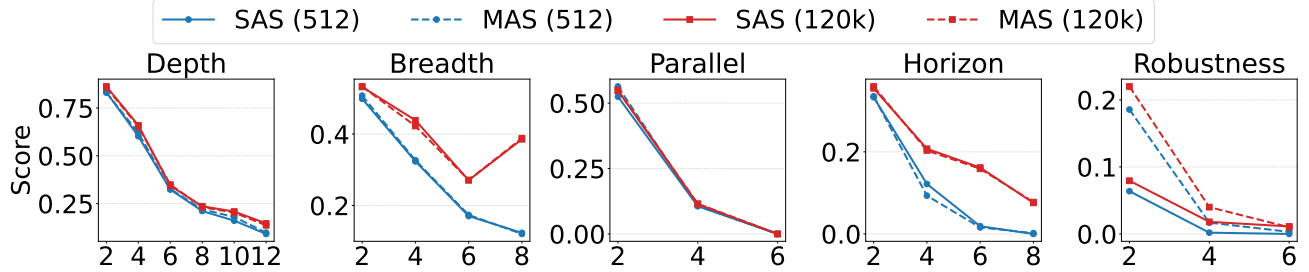


图 C.7. 不同最大上下文长度下的 Avg@8 对比，其中编排器为 Qwen-7b，子智能体为 GPT-120b。

D. 公共基准评估的补充讨论

D.1. 子智能体与基线设置

对于 SearchAgent，我们修改 OpenDeepResearch 代码⁴，在 HotpotQA 上使用 DuckDuckGo 搜索⁵，在 BrowseComp+ 上使用 BM25 检索器 (Robertson, 1994)。Table D.1 汇总了各子智能体，包括 MAS-Orchestra 中的编排决策（即需要由编排器生成的配置）及其对应实现。

对于基线系统，我们均使用原作者发布的官方代码运行。

⁴https://github.com/langchain-ai/open_deep_research

⁵<https://duckduckgo.com/>

编排器	GPT-20b (low)	Qwen-7b
问题	每个旧金山金门公园食蚁兽的数量等于 21。每个巴黎布洛涅森林袋鼠的数量等于每个墨西哥城查普尔特佩克公园狼的数量。每个巴黎布洛涅森林狼的数量等于 6。……墨西哥城查普尔特佩克公园有多少只驼鹿？	
思考过程	分析：我们需要设计智能体。问题是求墨西哥城查普尔特佩克公园的驼鹿数量。先快速解析。 变量定义如下： GGP_SF_Anteater = 21 (已知) GGP_SF_Wolf = 12 GGP_SF_Moose = ? GGP_SF_Kangaroo = ? GGP_SF_Moose 等于每个墨西哥城查普尔特佩克公园的狼。因此 GGP_SF_Moose = CP_MX_Wolf。 ... 没有其他约束涉及 CP_MX_Creatures，因此不存在问题。 因此答案为 42。 可以只创建一个智能体吗？单个智能体能够求解，但我们或许希望使用多智能体展示推理。不过约束允许单智能体，也可以为演示采用多智能体。 更简单的方案是使用一个输入为空的 CoTAgent 直接求解。这最为简洁，但必须确保得到最终答案。提供一个输入为空的最终智能体即可产生最终答案。 是否需要展示中间步骤？没有必要。 既然可以使用单智能体，那就使用单智能体。 ... 开始生成。 最终回答 <thinking> 该问题是一个简单的线性方程组，唯一关心的未知量是墨西哥城查普尔特佩克公园的驼鹿数量。所有其他变量要么直接给出，要么可以用已知常数表示。单个 CoTAgent 即可解析陈述、实例化变量、执行必要代入，并给出最终数值答案。由于不需要中间智能体，该图只包含这一个汇点智能体，它同时也是起始节点。	<thinking> 该问题需要建立并求解一个方程组。使用单个智能体并不充分，因为它会混合问题解析与求解步骤，增加出错风险。我们需要先把问题解析为方程，再求解方程组，最后验证解。这样的分工能保证准确性，并允许独立验证每一步。最终答案将由 Final_Answer 智能体给出。
执行输出	21	19
真实答案	19	

表 C.2. 指令微调 LLM (Qwen-7b) 与 RLM (GPT-20b (low)) 在同一问题实例上的对比。

D.2. 总体结果的补充观察

Section 6 报告了 MAS-Orchestra 相对于所有基线的优势。本节基于我们运行这些系统的经验给出补充观察。

推理时编排基线。 推理时编排方法的计算成本往往很高，AFlow 与 MAS-Zero 尤其如此。正如原论文 (Ket et al., 2025b) 所指出的，MAS-Zero 需要参数量超过 32B 的编排器才能产生有意义的结果。AFlow 是推理时方法中表现最强的基线，但仍明显逊于 MAS-Orchestra。

子智能体	编排决策	实现
CoTAgent	输入	Section G.3
SCAgent	输入	Section G.3
ReflexionAgent	输入	Section G.3
DebateAgent	输入、辩论角色	Section G.3
SearchAgent (HotpotQA)	输入	自主式 (多轮 + DuckDuckGoSearch)
SearchAgent (BrowseComp+)	输入	自主式 (多轮 + BM25)

表 D.1. 子智能体汇总。

基于训练的编排基线。 基于训练的编排基线出人意料地逊于许多推理时方法。我们观察到，当无法生成有效 MAS 时，MAS-GPT 经常回退到自洽 (SC)；SC 本身就是一个很强的单智能体系统。与 ToolOrchestra 相比，这种回退行为对 MAS-GPT 报告的性能优势贡献很大。

我们还观察到，ToolOrchestra 经常拒绝调用任何工具。在 ToolOrchestra 中，子智能体被视为工具；当子智能体解决任务的能力强于编排器时，拒绝调用会造成性能下降。相比之下，MAS-Orchestra 不依赖任何人工设计的回退策略，所有 MAS 都由学习得到的编排器直接生成。在 DoM 表述下，MAS-Orchestra 中的子智能体始终得到有效利用，如图 D.1、图 D.3和图 D.2所示。

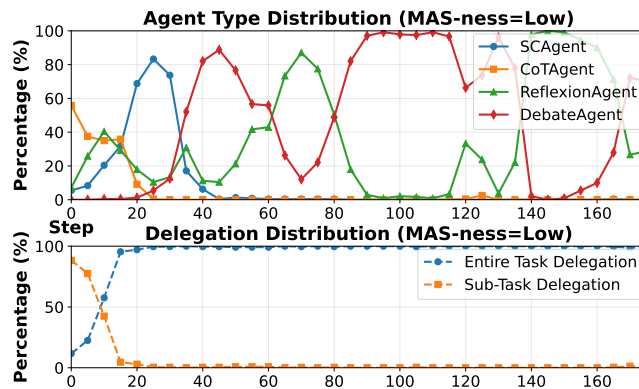


图 D.1. 低 DoM 下的智能体统计 (AIME24)。

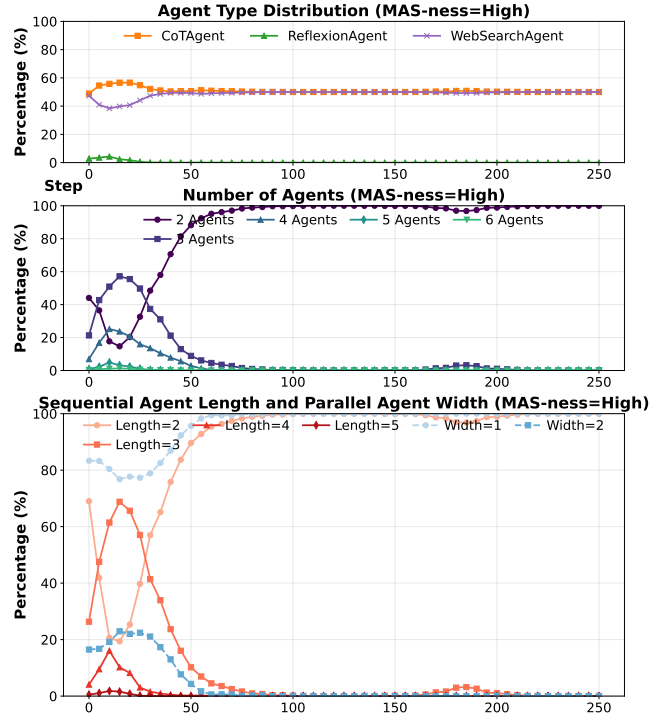


图 D.2. 高 DoM 下的智能体统计 (HotpotQA)。

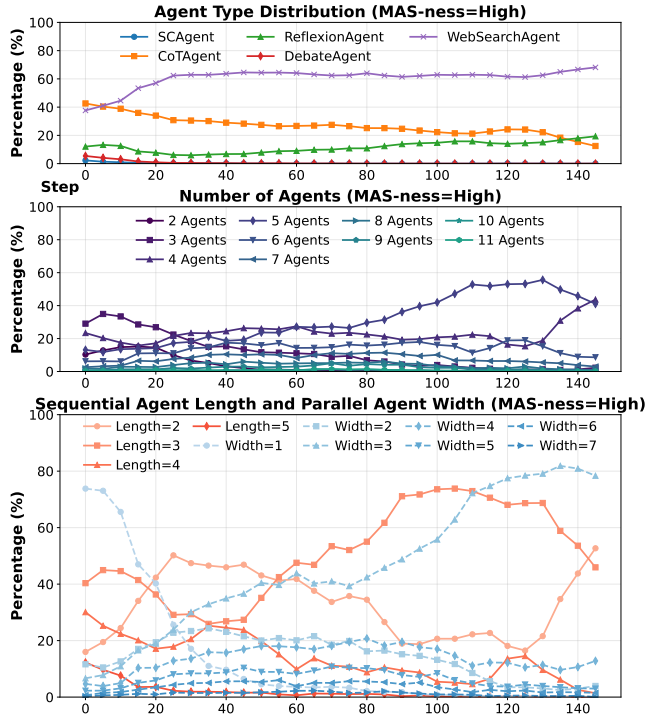


图 D.3. 高 DoM 下的智能体统计 (BrowseComp+)。

D.3. 基准与生成的 MAS

除 Section 6 报告的结果外，我们还在 Figure D.2 中给出了 HotpotQA 的统计信息。对于 HotpotQA，尽管问题可能允许并行搜索，编排器往往只使用一个 SearchAgent（再加上最终的 CoTAgent，共两个智能体）。这说明这些问题相对简单，一次搜索便足以解决。

结合图 D.1 中的 AIME 结果与图 D.3 中的 BrowseComp+ 结果，这些观察表明 MAS-Orchestra 能够动态适应给定任务：它会提出与底层子任务结构相符的 MAS 设计，并把执行委派给最有效的智能体配置。需要指出的是，由于底层 LLM 能力各异，生成的 MAS 设计不一定与底层任务结构完全一致（例如 HotpotQA）；这恰恰凸显了它相对于人工设计编排策略的优势。

D.4. 成本比较

Figure 1 展示了成本-性能权衡，Table D.2 则报告了 AIME24 与 GPQA 上相应的详细推理成本。整体式编排预计会比顺序式编排系统更高效，相较推理时编排方法的优势则更为明显。与这一直觉一致，按 LLM 调用次数、词元数量（包括提示词元和补全文本词元）与总成本计算，MAS-Orchestra 的效率提升均超过 $10\times$ （成本基于撰写本文时 Together AI 的定价）。我们还观察到明显更快的实际运行时间。需要注意的是，实际运行时间不仅反映算法设计，也取决于实现细节；我们进行了大量优化（例如异步执行子智能体），并将向社区发布完整代码库。

方法	AIME24				GPQA			
	调用	词元 (M)	成本 (\$)	时间 (ks)	调用	词元 (M)	成本 (\$)	时间 (ks)
CoTAgent	30	0.05	0.03	0.38	198	0.16	0.06	1.18
SCAgent	156	0.25	0.13	1.88	990	0.80	0.29	5.43
DebateAgent	332	0.37	0.13	2.29	2178	2.04	0.55	7.65
ReflexionAgent	112	0.14	0.06	0.67	518	0.44	0.13	2.50
MaAS	501	1.28	0.62	6.96	2936	7.52	3.11	39.52
AFlow	1288	3.47	1.46	19.28	7404	12.38	4.52	56.94
MAS-GPT	228	0.74	0.37	5.47	1516	3.90	1.76	29.56
ToolOrchestra	1319	3.77	1.76	31.35	7829	15.38	5.79	92.47
MAS-Orchestra	51	0.03	0.01	0.50	261	0.22	0.07	0.46

表 D.2. AIME24 与 GPQA 上的推理时统计。词元数以百万 (M) 为单位，实际运行时间以千秒 (ks) 为单位。

E. 推理时编排的补充相关工作

近期的自动化 MAS 设计工作通常在推理时运行，并依赖验证集指导适应。MASS (Zhou et al., 2025) 使用拒绝采样选择有效的智能体配置，MaAS (Zhang et al., 2025a) 则在 MASS 基础上引入逐问题掩码机制来适应子网络。ADAS (Hu et al., 2025) 与 AFlow (Zhang et al., 2025c) 将 MAS 设计视为代码生成任务：ADAS 根据验证性能存储并搜索历史设计，AFlow 则进一步使用蒙特卡洛树搜索。MAS-Zero (Ke et al., 2025b) 是一个例外，它完全在推理时执行自动化 MAS 设计，不依赖验证集。然而，这些推理时方法的适应能力有限，因为不可靠的验证信号或 LLM 评判器的错误指导都可能使适应过程偏离正确方向。

F. Mas-Orchestra 中的策略优化

通过最大化下式来优化 Equation (4) 中的策略：

$$J_{\text{GRPO}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, \{a_i\}_{i=1}^K \sim \pi_{\theta_{\text{old}}}(\cdot | x, m)} \frac{1}{K} \sum_{i=1}^K \frac{1}{|a_i|} \sum_{t=1}^{|a_i|} \left\{ \min \left[r_{i,t}(\theta) \hat{A}_{i,t}, \text{clip} \left(r_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{i,t} \right] - \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}}) \right\}. \quad (5)$$

其中

$$r_{i,t}(\theta) = \frac{\pi_{\theta}(a_{i,t} \mid x, a_{i,<t}, m)}{\pi_{\theta_{\text{old}}}(a_{i,t} \mid x, a_{i,<t}, m)}. \quad (6)$$

对于结果监督训练，我们为 a_i 中的所有词元分配序列级、组归一化的优势：

$$\hat{A}_{i,t} = \hat{R}_i, \quad \hat{R}_i = \frac{R_i - \text{mean}(\{R_j\}_{j=1}^K)}{\text{std}(\{R_j\}_{j=1}^K)}. \quad (7)$$

其中， $\bar{R} = \frac{1}{K} \sum_{i=1}^K R_i$ 是组内平均奖励， θ_{old} 表示用于生成样本的策略参数。

G. 编排器提示词

G.1. 低 DoM

系统提示词

你是一名乐于助人的助手。

DoM（MAS 程度）：低

有效通道：thinking、agent、answer

模型：[MODEL]

智能体是预先配置的 AI 角色，可以接受委派的任务。每个子智能体：

1. 具有特定用途与专业领域；
2. 使用独立于主对话的上下文窗口；
3. （可选）可配置允许其使用的特定工具；
4. 包含用于指导其行为的自定义系统提示词。

只能在 `<thinking>` 通道内调用智能体，并使用 `<agent>` 标签定义智能体。每个智能体都必须包含 `<agent_name>`、`<agent_description>`、`<required_arguments>` 和 `<agent_output_id>`。

不得遗漏任何请求字段，并确保回答是格式良好的 XML 对象！

开发提示词

通道：

`<thinking>`：内部推理与规划

`<agent>`：定义子智能体

`<answer>`: 面向用户的最终答案

模型（子智能体使用的模型）:

gpt-4.1-nano: [官方网站上的模型介绍]

gpt-oss-120b: [官方网站上的模型介绍]

(……列出所有候选模型)

DoM 级别:

low: 直接求解或至多使用一个智能体

high: 复杂的多智能体委派

子智能体模式（所有字段均为必需）:

`<agent>`

`<agent_name> ... </agent_name>` (从 CoTAgent、SCAgent、DebateAgent、ReflexionAgent 中选择一个)

`<agent_description> ... </agent_description>`

`<required_arguments>` (确保设置所有必需参数, 必须遵循 XML 格式)

`<...>...</...>`

`<...>...</...>`

`</required_arguments>`

`<agent_output_id> ... </agent_output_id>`

`</agent>`

可用智能体:

CoT: [描述、名称、必需参数和实现摘要]

SC: [描述、名称、必需参数和实现摘要]

Debate: [描述、名称、必需参数和实现摘要]

Reflexion: [描述、名称、必需参数和实现摘要]

Search: [描述、名称、必需参数和实现摘要]

用户提示词

请逐步解决该问题。在推理过程中, 你可以自行处理任务, 并在 `<answer>` 标签中输出最终答案; 也可以把任务委派给你设计的智能体, 再在 `<answer>` 标签中输出对应的 `agent_output_id`。你必须输出所有必需参数, 并为智能体使用完全一致的字段名。

例如,

如果你能自行解决任务, 应输出:

```

<thinking>
(20+9)*(30+7) = 600 + 140 + 270 + 63 = 1073.
</thinking>
<answer>1073</answer>

```

如果你能通过一次工具调用、委派给单个智能体来解决任务，应输出：

```

<thinking>
该问题涉及符号积分以及微积分基本定理的应用。它需要结构化推理，而非简单的数值计算。我将使用
一个能够执行逐步思维链推理的微积分智能体。原问题的最终答案将采用 CoTAgent 的输出。
</thinking>
<agent>
  <agent_name>CoTAgent</agent_name>
  <agent_description>通过一次思维链调用求解定积分。</agent_description>
  <required_arguments>
    <agent_input></agent_input>
  </required_arguments>
  <agent_output_id> calc_agent_output </agent_output_id>
</agent>
<answer>calc_agent_output</answer>

```

另一个示例：

```

<thinking>
该问题需要比较两个非常接近的数值选项。为确保准确，我将让两个推理角色展开辩论：一个关注数学
精度，另一个关注实际取整。DebateAgent 能同时考虑两种观点，并得出有依据的最终答案。最终答案将
采用 DebateAgent 的输出。
</thinking>
<agent>
  <agent_name>DebateAgent</agent_name>
  <agent_description> 通过一次辩论调用解决数值极为接近的选择问题。</
agent_description>
  <required_arguments>
    <agent_input></agent_input>
    <debate_roles> [" 数学教授", " 统计学教师"] </debate_roles>
  </required_arguments>
  <agent_output_id> compare_agent_output </agent_output_id>
</agent>

```

```
<answer>compare_agent_output</answer>
```

更多示例：

```
<thinking>
```

可以直接计算 17^3 ，但很容易出现算术错误。采用带自洽性的思维链 (CoT_SC)，可以采样多条推理路径并合并结果，从而提高准确性。原问题的最终答案将采用 SCAgent 的输出。

```
</thinking>
```

```
<agent>
```

```
  <agent_name>SCAgent</agent_name>
```

```
  <agent_description> 使用带自洽性的思维链 (CoT_SC) 执行算术计算。</agent_description>
```

```
  <required_arguments>
```

```
    <agent_input></agent_input>
```

```
  </required_arguments>
```

```
  <agent_output_id>math_agent_output</agent_output_id>
```

```
</agent>
```

```
<answer>math_agent_output</answer>
```

最后一个示例：

```
<thinking>
```

该任务需要使用公式进行推理，并确保单位处理正确。我将使用 Reflexion 智能体；如果出现错误，它可以反思并改进自己的推理。原问题的最终答案将采用 ReflexionAgent 的输出。

```
</thinking>
```

```
<agent>
```

```
  <agent_name>ReflexionAgent</agent_name>
```

```
  <agent_description> 使用 Reflexion 的迭代自我反思与批评来解决推理任务。</agent_description>
```

```
  <required_arguments>
```

```
    <agent_input></agent_input>
```

```
  </required_arguments>
```

```
  <agent_output_id>reflexion_agent_output</agent_output_id>
```

```
</agent>
```

```
<answer>reflexion_agent_output</answer>
```

如果把一个直接计算步骤与一次 Reflexion 工具调用结合，应输出：

```
<thinking>
```

步骤 1：手工计算平均值： $\text{sum} = 2 + 3 + 5 + 7 + 11 = 28$ $\text{avg} = 28 / 5 = 5.6$

步骤 2: 剩余步骤是把 $\text{sqrt}(5.6)$ 计算到小数点后三位, 这需要精确计算与数值改进。我会把这一部分委派给 ReflexionAgent; 若取整错误, 它将进行自我纠正。原问题的最终答案将采用 ReflexionAgent 的输出。

```
</thinking>
<agent>
  <agent_name>ReflexionAgent</agent_name>
  <agent_description> 通过轻量自我改进循环计算平方根（单次智能体调用） 。</
agent_description>
  <required_arguments>
    <agent_input>将  $\text{sqrt}(5.6)$  计算到小数点后三位</agent_input>
  </required_arguments>
  <agent_output_id>numeric_agent_output</agent_output_id>
</agent>
<answer>numeric_agent_output</answer>
```

下面是需要解决的问题:

[QUESTION]

G.2. 高 DoM

系统提示词

你是一名乐于助人的助手。

DoM (MAS 程度): 高

有效通道: thinking、agent、edge

模型: [MODEL]

智能体是预先配置的 AI 角色, 可以接受委派的任务。每个子智能体:

1. 具有特定用途与专业领域;
2. 使用独立于主对话的上下文窗口;
3. (可选) 可配置允许其使用的特定工具;
4. 包含用于指导其行为的自定义系统提示词。

只能在 `<thinking>` 通道内调用智能体, 并使用 `<agent>` 标签定义智能体。每个智能体都必须包含 `<agent_id>`、`<agent_name>`、`<agent_description>` 和 `<required_arguments>`。若要连接多个智能体以构成多智能体系统, 请使用 `<edge>` 通道。

不得遗漏任何请求字段, 并确保回答是格式良好的 XML 对象!

开发提示词

通道:

`<thinking>`: 内部推理与规划
`<agent>`: 定义子智能体
`<answer>`: 面向用户的最终答案

模型 (子智能体使用的模型):

`gpt-4.1-nano`: [官方网站上的模型介绍]
`gpt-oss-120b`: [官方网站上的模型介绍]
(……列出所有候选模型)

DoM 级别:

`low`: 直接求解或至多使用一个智能体
`high`: 复杂的多智能体委派

子智能体模式 (所有字段均为必需):

```
<agent>
  <agent_name> ... </agent_name> (从 CoTAgent、SCAgent、DebateAgent、ReflexionAgent 中选择一个)
  <agent_description> ... </agent_description>
  <required_arguments> (确保设置所有必需参数, 必须遵循 XML 格式)
    <...>...</...>
    <...>...</...>
  </required_arguments>
  <agent_output_id> ... </agent_output_id>
</agent>
```

边模式 (单个代码块; 所有字段均为必需。每一对标签定义一条有向连接: `<from>` 的输出 $\rightarrow$ `<to>` 的输入。请在此列出所有连接; 每个解决方案只能使用一个 `<edge>` 块):

```
<edge>
  <from> ... </from> (源智能体的 agent_id)
  <to> ... </to> (目标智能体的 agent_id)
</edge>
```

可用智能体:

CoT: [描述、名称、必需参数和实现摘要]

SC: [描述、名称、必需参数和实现摘要]
 Debate: [描述、名称、必需参数和实现摘要]
 Reflexion: [描述、名称、必需参数和实现摘要]
 Search: [描述、名称、必需参数和实现摘要]

用户提示词

请创建一个或多个智能体，并将它们连接成有效的计算图，通过协作解决给定问题并生成最终答案。创建智能体时，必须输出 `<agent>` 来定义它，其中包括：`<agent_id>`（智能体的唯一 ID，只能包含字母、数字或下划线，例如 A1、Refine_1、WS_Japan）、`<agent_name>`（必须严格为 CoTAgent、SCAgent、DebateAgent、ReflexionAgent 或 WebSearchAgent 之一）、`<agent_description>`、`<required_arguments>`（必须至少包含一个 `<agent_input>` 标签；DebateAgent 必须通过 `<debate_roles>` 定义两个或更多角色；如果 `<agent_input>` 留空（""），解析器会自动用原问题替换它）。

定义全部智能体后，必须通过指定描述智能体间数据流的边来构建有效图。只能输出一个 `<edge>` 块；每对 `<from>` 和 `<to>` 将一个智能体的输出连接到另一个智能体的输入：

```
<edge>
  <from>source_agent_id</from>
  <to>target_agent_id</to>
</edge>
```

可以在 `<edge>` 内输出多组 `<from>` 和 `<to>`。每个 `<from>` 与 `<to>` 的值都必须与已有的 `<agent_id>` 完全匹配。

要使图有效，必须满足以下全部约束：

- 节点一致性：每个 `<from>` 和 `<to>` 都必须引用出现在某个 `<agent>` 块中的有效 `<agent_id>`。
- 有向性：边是有向的，数据从 `<from>` 流向 `<to>`。
- 连通性：每个智能体都必须直接或间接连接到主流程，不允许存在孤立智能体。
- 起始节点：至少一个智能体没有入边，它们是“入口点”（例如 WebSearch 或初始推理）。
- 汇点：必须恰好有一个没有出边的智能体，它就是生成答案的最终智能体。
- 不得出现未定义边：`<from>` 或 `<to>` 不得引用尚未声明的智能体。
- 不得出现自环或环路：不允许 `<from>X</from><to>X</to>` 形式的自环；图必须是有向无环图，并且存在拓扑序。
- 允许并行：多个智能体可以具有相同的 `<from>` 或 `<to>`，即允许扇出和扇入。
- 汇点无歧义：解析器会拒绝包含多个汇点的图；必要时请增加一个最终的“收集器”智能体。
- 与顺序无关：XML 中各边的排列不必遵循执行顺序，系统会自动进行拓扑排序。
- 汇点答案完整性：唯一汇点智能体的输出必须以面向用户的形式直接回答原问题。除非问题明确要求，否则它不能只是中间产物（例如笔记、批评或原始表格）。如果汇点的 `<agent_input>` 为空，它会

继承原问题，并必须返回最终答案。如果 `<agent_input>` 非空，运行器仍会在其前面加入原问题作为上下文；汇点仍须给出面向用户的最终答案。

- 边与数据流双向一致：边表示执行顺序， $\{ \}$ 表示数据流，二者必须相互一致。（a）如果 `<agent_input>` 引用了 $\{X\}$ ，则必须存在边 `<from>X</from><to>该智能体</to>`。（b）如果存在边 `<from>X</from><to>Y</to>`，则 Y 的 `<agent_input>` 必须引用 $\{X\}$ 。换言之，当且仅当一个智能体确实向另一个智能体传递数据时，二者之间才应存在边。不得仅为规定执行顺序而创建没有数据流的边。

思考部分（必需）：

定义智能体和边之前，必须包含一个 `<thinking>` 部分。其中应自然说明为何需要多个智能体、为何选择各类智能体，以及图为何采用该结构（并行、顺序或混合）。它必须同时论证规划与设计理由。

示例结构：

`<thinking>` 解释单个智能体为何不足。描述每个智能体的角色及其连接方式。论证流程模式（并行、顺序或混合）。最后明确说明由哪个智能体生成最终输出。`</thinking>`

单智能体示例：

如果决定使用单个智能体求解，应按如下格式输出。在这种情况下，由于 `<agent_input>` 与原任务相同，必须把 `<agent_input>` 设为空（""），解析器会用原问题替换它。

示例 1：

问题：计算 $(2x + 5) dx$ 在 0 到 3 上的定积分。

`<thinking>`

该问题涉及符号积分以及微积分基本定理的应用。它需要结构化推理，而非简单的数值计算。我将使用一个能够执行逐步思维链推理的微积分智能体。原问题的最终答案将采用 CoTAgent 的输出。

`</thinking>`

`<agent>`

`<agent_id>calc_agent</agent_id>`

`<agent_name>CoTAgent</agent_name>`

`<agent_description>` 通过一次思维链调用求解定积分。`</agent_description>`

`<required_arguments>`

`<agent_input></agent_input>`

`</required_arguments>`

`</agent>`

示例 2:

问题: 17 的立方是多少?

<thinking>

可以直接计算 17^3 , 但很容易出现算术错误。采用带自洽性的思维链 (CoT_SC), 可以采样多条推理路径并合并结果, 从而提高准确性。原问题的最终答案将采用 SCAgent 的输出。

</thinking>

<agent>

<agent_id>math_agent</agent_id>

<agent_name>SCAgent</agent_name>

<agent_description> 使用带自洽性的思维链 (CoT_SC) 执行算术计算。</agent_description>

<required_arguments>

<agent_input></agent_input>

</required_arguments>

</agent>

示例 3:

问题: 已知 $x^2 = 46.694444$, 目标数 45 和 46 中哪一个更接近?

<thinking>

该问题需要比较两个非常接近的数值选项。为确保准确, 我将让两个推理角色展开辩论: 一个关注数学精度, 另一个关注实际取整。DebateAgent 能同时考虑两种观点, 并得出有依据的最终答案。最终答案将采用 DebateAgent 的输出。

</thinking>

<agent>

<agent_id>compare_agent</agent_id>

<agent_name>DebateAgent</agent_name>

<agent_description> 通过一次辩论调用解决数值极为接近的选择问题。</agent_description>

<required_arguments>

<agent_input></agent_input>

<debate_roles>["数学教授", "统计学教师"]</debate_roles>

</required_arguments>

</agent>

示例 4:

问题：一列火车以每小时 60 英里的速度行驶，2.5 小时能行驶多远？

<thinking>

该任务需要使用公式进行推理，并确保单位处理正确。我将使用 Reflexion 智能体；如果出现错误，它可以反思并改进自己的推理。原问题的最终答案将采用 ReflexionAgent 的输出。

</thinking>

<agent>

<agent_id>reflexion_agent</agent_id>

<agent_name>ReflexionAgent</agent_name>

<agent_description> 使用 Reflexion 的迭代自我反思与批评来解决推理任务。</agent_description>

<required_arguments>

<agent_input></agent_input>

</required_arguments>

</agent>

示例 5:

问题：截至本月，日本当前的通货膨胀率是多少？

<thinking>

该问题依赖最新事实信息，无法仅凭静态知识可靠回忆。使用单个 WebSearchAgent 即可，因为该任务只需从网络检索准确且带引用的事实，无需进一步推理或综合。该智能体将搜索在线来源，并返回简洁、基于引用的摘要。因此，WebSearchAgent 既是流程中的唯一节点，也是最终输出节点。

</thinking>

<agent>

<agent_id>SEARCH</agent_id>

<agent_name>WebSearchAgent</agent_name>

<agent_description> 从互联网检索最新且带引用的事实信息。</agent_description>

<required_arguments>

<agent_input></agent_input>

</required_arguments>

</agent>

多智能体示例:

如果决定使用多个智能体求解，必须先把原问题分解为更小且定义明确的子任务，每个子任务只对应一个子目标。随后为每个子任务创建一个智能体，并将它们连接成连贯的无环计算图。

在这种情况下，agent_input 不为空，而是智能体需要解决的具体子任务；解析器会自动把原问题作为上下文添加到给定 agent_input 内容之前。

分解时：

1. 保持子任务最小且聚焦。每个智能体应只处理一个原子目标（例如一次查询、一个推理步骤或一项验证任务）。
2. 针对不同事实证据使用多个 WebSearchAgent，而不是执行一次宽泛搜索。
3. 按逻辑连接智能体，使信息流向直接回答原问题的唯一最终智能体（汇点）。

示例 1：

问题：简要总结纽约市、洛杉矶和芝加哥最新的住房空置率，并附引用。

<thinking>

我们需要带来源的最新数据，因此将并行运行三个 WebSearch 智能体。随后，由一个 CoT 智能体把三个片段规范化为小表格。一个 SC 智能体将运行多个细微变体并投票，以减少抽取错误。一个 Reflexion 智能体将检查单位、时效性与引用。最后由一个 CoT 智能体撰写简短摘要。最终汇点是 FINAL。

</thinking>

<agent>

<agent_id>WS_NYC</agent_id>

<agent_name>WebSearchAgent</agent_name>

<agent_description> 查找纽约市最新住房空置率及来源文本。</agent_description>

<required_arguments>

<agent_input> 搜索纽约市住房空置率的最新官方或可靠估计。返回包含数值、日期和一行引用的简短片段。</agent_input>

</required_arguments>

</agent>

<agent>

<agent_id>WS_LA</agent_id>

<agent_name>WebSearchAgent</agent_name>

<agent_description> 查找洛杉矶最新空置率及来源文本。</agent_description>

<required_arguments>

<agent_input> 搜索洛杉矶住房空置率的最新官方或可靠估计。返回包含数值、日期和一行引用的简短片段。</agent_input>

</required_arguments>

</agent>

<agent>

<agent_id>WS_CHI</agent_id>

```

<agent_name>WebSearchAgent</agent_name>
<agent_description> 查找芝加哥最新空置率及来源文本。</agent_description>
<required_arguments>
  <agent_input> 搜索芝加哥住房空置率的最新官方或可靠估计。返回包含数值、日期和一行引用的简短片段。</agent_input>
</required_arguments>
</agent>
<agent>
  <agent_id>EXT</agent_id>
  <agent_name>CoTAgent</agent_name>
  <agent_description> 抽取数值，并统一三个空置率的日期与引用格式。</agent_description>
  <required_arguments>
    <agent_input> 从以下片段中抽取每座城市的名称、空置率（百分比）、参考日期（YYYY-MM 或 YYYY）和简短引用。输出一张紧凑的三行表格。
    纽约市: ${WS_NYC}
    洛杉矶: ${WS_LA}
    芝加哥: ${WS_CHI} </agent_input>
  </required_arguments>
</agent>
<agent>
  <agent_id>VOTE</agent_id>
  <agent_name>SCAgent</agent_name>
  <agent_description> 对抽取结果进行集成，以减少复制或解析错误。</agent_description>
  <required_arguments>
    <agent_input> 给定以下三行表格，独立执行五次抽取，并投票得到一份纠正后的三行表格。
    表格: ${EXT} </agent_input>
  </required_arguments>
</agent>
<agent>
  <agent_id>QA</agent_id>
  <agent_name>ReflexionAgent</agent_name>
  <agent_description> 检查单位、日期时效性与引用是否存在；必要时列出修正项。</agent_description>
  <required_arguments>
    <agent_input> 审核投票后的表格：单位应为%，日期应存在，并且每座城市都应有引用。如发现问题，用三行列出具体修正；否则回答 OK。最后给出一行结论。
    投票后的表格: ${VOTE} </agent_input>
  </required_arguments>
</agent>

```

```

    </required_arguments>
</agent>
<agent>
  <agent_id>FINAL</agent_id>
  <agent_name>CoTAgent</agent_name>
  <agent_description> 撰写带引用的简短摘要。</agent_description>
  <required_arguments>
    <agent_input> 使用检查后的表格与注释，撰写 3-4 句话的摘要：每座城市一句，最后一句比
较各空置率。保留表格中的行内来源引用。
表格：${VOTE}
质量检查注释：${QA} </agent_input>
  </required_arguments>
</agent>
<edge>
  <from>WS_NYC</from><to>EXT</to>
  <from>WS_LA</from><to>EXT</to>
  <from>WS_CHI</from><to>EXT</to>
  <from>EXT</from><to>VOTE</to>
  <from>VOTE</from><to>QA</to>
  <from>VOTE</from><to>FINAL</to>
  <from>QA</from><to>FINAL</to>
</edge>

```

示例 2:

问题：20 世纪 70 年代末，教皇约翰·保罗二世首次出访时到访了一个以丰富的中美洲历史和庞大人口著称的国家。后来他在哪一天访问了一座亚得里亚海沿岸的重要城市？该城市以重要港口以及一座供奉姓名首字母为“S”的圣人的著名宗座圣殿而闻名；同一天他还访问了附近的哪座城市？

```

<thinking>
该问题需要历史推理：先识别教皇约翰·保罗二世的首次出访，再找出他访问过的一座亚得里亚海城市，
该市拥有供奉姓名以“S”开头圣人的宗座圣殿；随后确定访问日期，以及他同日访问的另一座城市。
单个智能体无法完成全部工作，因为任务混合了检索与推理。我将把它分解为更小的子任务：三个
WebSearchAgent 分别检索事实，一个 CoTAgent 构建时间线，一个 ReflexionAgent 验证一致性，最后一个
CoTAgent 撰写答案。由 FINAL 智能体输出最终答案。
</thinking>
<agent>
  <agent_id>WS_FIRST_TRIP</agent_id>
  <agent_name>WebSearchAgent</agent_name>

```

```

    <agent_description> 检索教皇约翰·保罗二世在 20 世纪 70 年代末首次出访的细节。</
agent_description>
    <required_arguments>
        <agent_input> 搜索教皇约翰·保罗二世的首次出访（年份、目的国和持续时间）。返回行程日
期范围、目的地和可靠引用。</agent_input>
    </required_arguments>
</agent>
<agent>
    <agent_id>WS_ADRIATIC</agent_id>
    <agent_name>WebSearchAgent</agent_name>
    <agent_description> 查找亚得里亚海沿岸城市的访问记录，其中城市拥有供奉姓名以“S”开头
圣人的宗座圣殿。</agent_description>
    <required_arguments>
        <agent_input> 搜索教皇约翰·保罗二世访问过的亚得里亚海城市，该市须有一座供奉姓名首
字母为“S”的圣人的宗座圣殿（例如圣尼古拉或圣马可）。返回城市名、宗座圣殿名、访问日期与引用。</
agent_input>
    </required_arguments>
</agent>
<agent>
    <agent_id>WS_SAME_DAY</agent_id>
    <agent_name>WebSearchAgent</agent_name>
    <agent_description> 识别教皇约翰·保罗二世在访问该亚得里亚海城市当天还到访过的附近
城市。</agent_description>
    <required_arguments>
        <agent_input> 搜索教皇约翰·保罗二世在访问该亚得里亚海城市同一天到访的其他城市。返
回附近城市名、距离估计与来源引用。</agent_input>
    </required_arguments>
</agent>
<agent>
    <agent_id>TIMELINE</agent_id>
    <agent_name>CoTAgent</agent_name>
    <agent_description> 把行程数据整合为一条经过验证的历史时间线。</
agent_description>
    <required_arguments>
        <agent_input> 使用以下结果：
首次出访：${WS_FIRST_TRIP}
亚得里亚海城市访问：${WS_ADRIATIC}
同日访问：${WS_SAME_DAY}
构建清晰的时间线，确认亚得里亚海城市的访问日期与同日到访的附近城市。</agent_input>

```

```

    </required_arguments>
  </agent>
  <agent>
    <agent_id>VERIFY</agent_id>
    <agent_name>ReflexionAgent</agent_name>
    <agent_description> 验证时间顺序、地理位置以及圣人与宗座圣殿之间的联系。</
agent_description>
    <required_arguments>
      <agent_input> 检查时间线事实是否一致：- 确保所有来源中出现的日期相同。- 确认亚得里亚
海城市在地理上邻近第二座城市。- 验证该宗座圣殿确实供奉一位姓名以“S”开头的圣人。
若信息一致则返回 OK，否则列出修正项。
待验证时间线：${TIMELINE} </agent_input>
    </required_arguments>
  </agent>
  <agent>
    <agent_id>FINAL</agent_id>
    <agent_name>CoTAgent</agent_name>
    <agent_description> 生成包含日期和两座城市的简洁最终答案。</agent_description>
    <required_arguments>
      <agent_input> 使用已验证的结果，以一句话回答：
“教皇约翰·保罗二世于 [DATE] 访问了 [ADRIATIC_CITY]，这里有供奉圣人 [S] 的宗座圣殿；同一天，
他还访问了 [NEARBY_CITY]。”
另附一句引用来源的简短验证说明。
时间线：${TIMELINE}
验证：${VERIFY} </agent_input>
    </required_arguments>
  </agent>
</edge>
  <from>WS_FIRST_TRIP</from><to>TIMELINE</to>
  <from>WS_ADRIATIC</from><to>TIMELINE</to>
  <from>WS_SAME_DAY</from><to>TIMELINE</to>
  <from>TIMELINE</from><to>VERIFY</to>
  <from>VERIFY</from><to>FINAL</to>
  <from>TIMELINE</from><to>FINAL</to>
</edge>

```

(……更多示例)

下面是需要解决的问题：

[QUESTION]

G.3. 子智能体

CoTAgent

```

async def CoTAgent(self, agent_input, model: str):
    # 基本设置
    temperature = 0.5
    # 思维链指令
    cot_instruction = " 请逐步思考，然后解决任务。"
    # 实例化 CoT LLM
    cot_agent = LLMAGENTBase([thinking, answer], 思维链 LLM, model=model, temperature=temperature)
    thinking, answer = await cot_agent([agent_input], cot_instruction)
    final_answer = self.make_final_answer(thinking, answer)
    return final_answer

func_string = inspect.getsource(CoTAgent)

COT = {
    "description": " 通过鼓励 LLM 逐步思考而非直接输出答案，思维链推理借助中间步骤解决复杂问题。"
    "name": " 思维链智能体 (CoTAgent) "
    "required_arguments": {
        "agent_input": "CoTAgent 的输入。若为空，解析器会用原问题替换。"
    }
    "implementation": "{func_string}"
}

```

SCAgent

```

async def SCAgent(self, agent_input, model: str):
    # 基本设置
    temperature = 0.5
    num_repeated_samples = 5
    # 思维链指令
    cot_instruction = " 请逐步思考，然后解决任务。"
    # 初始化多个 CoT 智能体以实现自治
    cot_agents = [ LLMAGENTBase([thinking, answer], 思维链 LLM, model=model, temperature=temperature) for _ in range(num_repeated_samples) ]
    thinking_mapping = {}
    answer_mapping = {}
    possible_answers = []
    for i in range(num_repeated_samples):
        thinking, answer = await cot_agents[i]([agent_input], cot_instruction)
        possible_answers.append(answer.content)
        thinking_mapping[answer.content] = thinking
        answer_mapping[answer.content] = answer
    # 通过多数投票集成答案
    answer = self.majority_voting(possible_answers)
    thinking = thinking_mapping[answer]

```



```

answer = answer_mapping[answer]
final_answer = self.make_final_answer(thinking, answer)
return final_answer

func_string = inspect.getsource(SCAgent)

COT_SC = {
    "description": "LLM 虽能得到正确答案，但其推理可能有所变化。通过以较高温度重复询问同一问题，可以生成多条推理路径；随后对思维链智能体的答案进行集成，得到更准确的最终答案。该方法最适合需要通过共识获得高置信度的问题。"
    "name": " 思维链自治智能体 (SCAgent) "
    "required_arguments": {
        "agent_input": "SCAgent 的输入。若为空 (""), 解析器会自动用原问题替换。"
    }
    "implementation": "{func_string}"
}

```

DebateAgent

```

async def DebateAgent(self, agent_input, model: str, debate_roles: List[str]):
    # 基本设置
    temperature = 0.5
    max_debate_round = 5
    # 初始推理指令
    debate_initial_instruction = " 请逐步思考，然后解决任务。"
    # 辩论更新指令
    debate_instruction = " 给定其他智能体的解答，将其观点视为建议，并给出更新后的答案。"
    # 使用不同角色初始化辩论智能体
    debate_agents = [ LLMAgentBase([thinking, answer], 辩论 LLM, model=model, role=role, temperature=temperature) for role in debate_roles ]
    # 最终决策指令
    final_decision_instruction = " 给定全部推理与答案，请谨慎给出最终答案。"
    final_decision_agent = LLMAgentBase([thinking, answer], 最终决策 LLM, model=model, temperature=temperature )
    all_thinking = [[] for _ in range(max_debate_round)]
    all_answer = [[] for _ in range(max_debate_round)]
    # 执行多轮辩论
    for r in range(max_debate_round):
        for i in range(len(debate_agents)):
            if r == 0:
                thinking, answer = await debate_agents[i]([agent_input], debate_initial_instruction)
            else:
                input_infos = [agent_input] + [all_thinking[r-1][i]] + all_thinking[r-1][:i] + all_thinking[r-1][i+1:]
                thinking, answer = await debate_agents[i](input_infos, debate_instruction)
            all_thinking[r].append(thinking)
            all_answer[r].append(answer)
    # 根据全部辩论结果作出最终决策
    thinking, answer = await final_decision_agent([agent_input] + all_thinking[max_debate_round-1] + all_answer[max_debate_round-1], final_decision_instruction )
    final_answer = self.make_final_answer(thinking, answer)

```

```

return final_answer

func_string = inspect.getsource(DebateAgent)

LLM_debate = {
    "description": " 通过让不同 LLM 相互辩论, 可以利用多元观点得到更好的解答。该智能体最适合能从多种视角中获益的问题。"
    "name": "LLM 辩论智能体 (DebateAgent) "
    "required_arguments": {
        "agent_input": "DebateAgent 的输入。若为空 (\"\"), 解析器会自动用原问题替换。"
        "debate_roles": " 代表不同视角的角色列表 (必须包含多个角色), 例如 “数学教授” 或 “统计学家”。"
    }
    "implementation": "{func_string}"
}

```

ReflexionAgent

```

async def ReflexionAgent(self, agent_input, model: str):
    # 基本设置
    temperature = 0.5
    max_reflection_round = 5
    # 初始推理指令
    initial_instruction = " 请逐步思考, 然后解决任务。"
    # 反思与改进指令
    reflect_instruction = " 给定之前的尝试与反馈, 请仔细思考最近一次尝试可能出错之处。利用此前尝试中的洞见, 更好地解决任务。"

    cot_agent = LLMAgentBase( [thinking, answer], 思维链 LLM, model=model, temperature=temperature )
    # 批评反馈指令
    critic_instruction = " 请审查以上答案, 并批评其中可能出错之处。如果你完全确定答案正确, 请在 'correct' 中严格输出 'True'。"
    critic_agent = LLMAgentBase( [feedback, correct], 批评 LLM, model=model, temperature=temperature )
    # 初始尝试
    cot_inputs = [agent_input]
    thinking, answer = await cot_agent(cot_inputs, initial_instruction, 0)
    # 迭代式自我反思循环
    for i in range(max_reflection_round):
        feedback, correct = await critic_agent( [agent_input, thinking, answer], critic_instruction, i )
        if correct.content == 'True':
            break
        cot_inputs.extend([thinking, answer, feedback])
        thinking, answer = await cot_agent(cot_inputs, reflect_instruction, i + 1)
    final_answer = self.make_final_answer(thinking, answer)
    return final_answer

func_string = inspect.getsource(ReflexionAgent)

Reflexion = {
    "description": " 为提升性能, LLM 可以根据反馈迭代改进答案。模型通过反思先前尝试并吸收批评来完善推理, 给出

```

更准确的解答。该智能体最适合能从自我纠正中获益的复杂问题。"

```
"name": " 自我改进 (Reflexion) "
"required_arguments": {
  "agent_input": "ReflexionAgent 的输入。若为空 (""), 解析器会自动用原问题替换。"
}
"implementation": "{func_string}"
}
```

H. 基于规则的解析器

H.1. 低 DoM

低 DoM 解析器

```
async def LowDoMParser( self, response_text, original_task_info):
    # 步骤 1: 检查是否指定了智能体
    agent_name = extract_xml(response_text, agent_name)
    if not agent_name:
        # 没有智能体: 回退到直接回答
        answer = extract_xml(response_text, answer)
        return direct_answer, answer

    # 步骤 2: 解析必需参数
    args_xml = extract_xml(response_text, required_arguments)
    required_args = {}

    # 处理 agent_input
    agent_input = extract_xml(args_xml, agent_input)
    if agent_input == "" or global_no_decompose:
        required_args[agent_input] = ""
    else:
        required_args[agent_input] = agent_input

    # 可选的 DebateAgent 参数
    debate_roles = extract_xml(args_xml, debate_roles)
    if agent_name.lower() == debateagent:
        roles = parse_list(debate_roles)
        assert len(roles) ≥ 2
        required_args[debate_roles] = roles

    # 步骤 3: 构造智能体调用
    agent_call = {
        "name": agent_name,
        "required_arguments": required_args
    }

    # 步骤 4: 生成统一的 forward() 分派器
    async def forward( self, original_task_info):
        agent_input = required_args[agent_input]
```

```

if agent_input == " ":
    task_info = original_task_info
else:
    task_info = CombineTask(original_task_info, agent_input)

# 根据智能体类型分派
if agent_name == SCAgent:
    return await self.SCAgent(task_info, model=global_node_model)
elif agent_name == CoTAgent:
    return await self.CoTAgent(task_info, model=global_node_model)
elif agent_name == ReflexionAgent:
    return await self.ReflexionAgent(task_info, model=global_node_model)
elif agent_name == DebateAgent:
    return await self.DebateAgent(task_info, model=global_node_model,
    debate_roles=required_args[debate_roles])
return forward

```

H.2. 高 DoM

高 DoM 解析器

```

async def HighDoMParser(self, response_text, original_task_info):
    # 步骤 1: 抽取全部智能体节点
    agents = ExtractAllAgents(response_text)
    if not agents:
        # 没有智能体: 回退到直接回答
        answer = extract_xml(response_text, answer)
        return direct_answer, answer

    # 步骤 2: 抽取边 (有向依赖)
    edges = ExtractEdges(response_text)

    # 步骤 3: 处理单智能体情况
    if len(agents) == 1 and len(edges) == 0:
        return GenerateSingleAgentForward(agents[0])

    # 步骤 4: 拒绝缺失边的无效多智能体情况
    if len(agents) > 1 and len(edges) == 0:
        raise ValueError(多个智能体必须通过边连接)

    # 步骤 5: 验证图为恰有一个汇点的有向无环图
    ValidateGraph(agents, edges)

    # 步骤 6: 计算执行顺序与汇点节点
    execution_order = TopologicalSort(agents, edges)
    sink = FindUniqueSink(agents, edges)

    # 步骤 7: 生成按图执行的多智能体 forward()
    return GenerateMultiAgentForward(agents, edges, execution_order, sink)

```

```

def ExtractAllAgents(response_text):
    agents = []
    # 查找所有 <agent>...</agent> 块
    for block in FindAllAgentBlocks(response_text):
        agent_id = extract_xml(block, agent_id)
        agent_name = extract_xml(block, agent_name)
        agent_desc = extract_xml(block, agent_description)
        args_xml = extract_xml(block, required_arguments)
        if not agent_id or not agent_name:
            continue

        required_args = { }
        agent_input = extract_xml(args_xml, agent_input)
        if agent_input == "" or not agent_input:
            required_args[agent_input] = ""
        else:
            required_args[agent_input] = agent_input

        debate_roles = extract_xml(args_xml, debate_roles)
        if debate_roles:
            roles = ParseList(debate_roles)
            if agent_name.lower() == debateagent:
                assert roles is not None
                assert len(roles) ≥ 2
                assert all(isinstance(r, str) and len(r) ≥ 2 for r in roles)
                required_args[debate_roles] = roles
            else:
                required_args[debate_roles] = roles

        agents.append( {
            "agent_id": agent_id,
            "agent_name": agent_name,
            "agent_description": agent_desc,
            "required_arguments": required_args
        })
    return agents

def ExtractEdges(response_text):
    edges = []
    edge_xml = extract_xml(response_text, edge)
    if not edge_xml:
        return edges

    # 按顺序读取每个 (from,to) 对
    for (u, v) in FindAllFromToPairs(edge_xml):
        edges.append((u.strip(), v.strip()))
    return edges

def ValidateGraph(agents, edges):

```

```

# 规则 1/6: 每个端点都必须是已声明的 agent_id
ids = set([a[agent_id] for a in agents])
for (u, v) in edges:
    if u not in ids or v not in ids:
        raise ValueError(边引用了未定义的智能体)

# 规则 4: 至少有一个起始节点 (无入边)
incoming = set([v for (u, v) in edges])
start_nodes = ids - incoming
if len(start_nodes) == 0:
    raise ValueError(没有起始节点)

# 规则 5/9: 恰有一个汇点节点 (无出边)
outgoing = set([u for (u, v) in edges])
sink_nodes = ids - outgoing
if len(sink_nodes) != 1:
    raise ValueError(必须恰有一个汇点节点)

# 规则 3: 连通性: 从起点可达, 且可以到达汇点
sink = list(sink_nodes)[0]
forward_reach = BFSForward(start_nodes, edges)
backward_reach = BFSBackward({sink}, edges)
connected = forward_reach and backward_reach
isolated = ids - (forward_reach & backward_reach)
if isolated:
    raise ValueError(存在孤立智能体)

# 规则 7: 必须无环 (DAG)
order = TopologicalSort(agents, edges)
if len(order) != len(agents):
    raise ValueError(检测到环路)

def TopologicalSort(agents, edges):
    # Kahn 算法 (入度为零节点的队列)
    ids = [a[agent_id] for a in agents]
    in_deg = {i: 0 for i in ids}
    adj = {i: [] for i in ids}
    for (u, v) in edges:
        adj[u].append(v)
        in_deg[v] += 1

    Q = Deque([i for i in ids if in_deg[i] == 0])
    order = []
    while Q:
        u = Q.popleft()
        order.append(u)
        for v in adj[u]:
            in_deg[v] -= 1
            if in_deg[v] == 0:

```



```

        Q.append(v)
    return order

async def GenerateMultiAgentForward( self, agents, edges, execution_order, sink):
    global_node_model = get_global(global_node_model)
    # 将 agent_id 映射到对应输出
    results = { }

    for agent_id in execution_order:
        cfg = Lookup(agents, agent_id)
        name = cfg[agent_name].lower()
        args = cfg[required_arguments]
        agent_input = args.get(agent_input, "")

        # 用先前结果替换 {id} 占位符
        if agent_input:
            for prev_id, prev_out in results.items():
                agent_input = ReplaceAll( agent_input, ${prev_id}, prev_out )

        # 准备 task_info: 始终包含原任务作为上下文
        if not agent_input:
            task_info = original_task_info
        else:
            combined = ( 原任务: + original_task_info.content + ; 当前子任务: + agent_input )
            task_info = Info(task, User, combined, ...)

        # 分派给选定的智能体
        if name == cotagent:
            out = await self.CoTAgent(task_info, model=global_node_model)
        elif name == scagent:
            out = await self.SCAgent(task_info, model=global_node_model)
        elif name == reflexionagent:
            out = await self.ReflexionAgent(task_info, model=global_node_model)
        elif name == debateagent:
            out = await self.DebateAgent( task_info, model=global_node_model, debate_roles=args[debate_roles] )
        elif name == websearchagent:
            out = await self.WebSearchAgent(task_info, model=global_node_model)
        else:
            raise ValueError(未知智能体类型)

    results[agent_id] = out

    return results[sink]

```